

Machine & Deep Learning pour l'Ingénierie des Données

Philip Tchatchoua

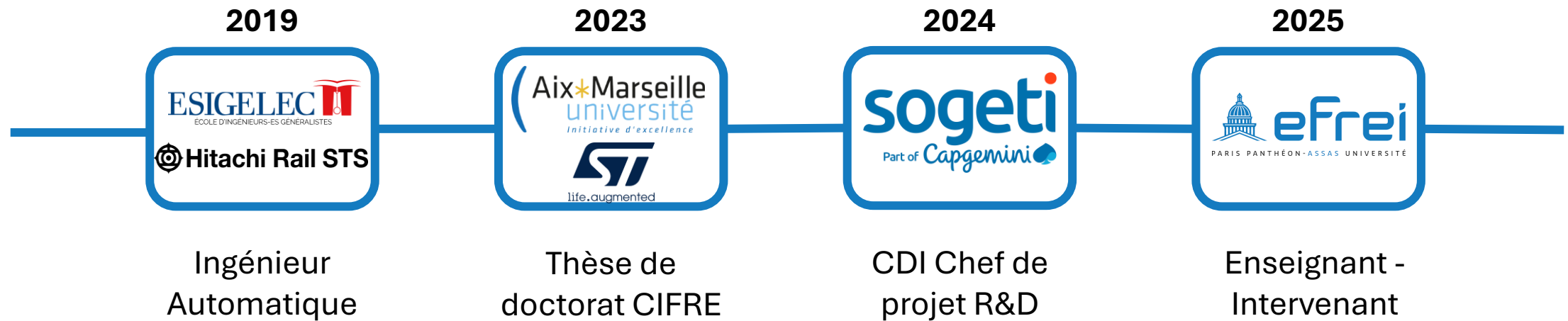
2025-2026



Mastère Data Engineering & IA

Hello, I'm Philip !

- Docteur en Automatique & Informatique
- Chercheur en Intelligence Artificielle
- Chef de projet R&D chez Sogeti
- Football / Mangas en loisirs



Et vous ?

- Nom et prénom
- Parcours scolaire en quelques mots
- Objectif professionnel cible
- Quelles attentes de la matière



Règles du cours



Ponctualité

Toujours signaler en amont tout retard / absence



Bavardage

Maintenir la salle de cours silencieuse



Participation active

Poser des questions, contribuer aux discussions



Téléphone

Sur silencieux et utilisation interdit durant le cours



Bienveillance

Faire preuve de respect et de solidarité



Langage

Eviter tout propos déplacé ou offensant



Travail personnel

Fournir un travail honnête et original



Mange

Ne pas manger ni boire pendant le cours

Modalités d'évaluation



Evaluation
écrite
25 %



Projet Machine
Learning
35 %



Projet Deep
Learning
40 %

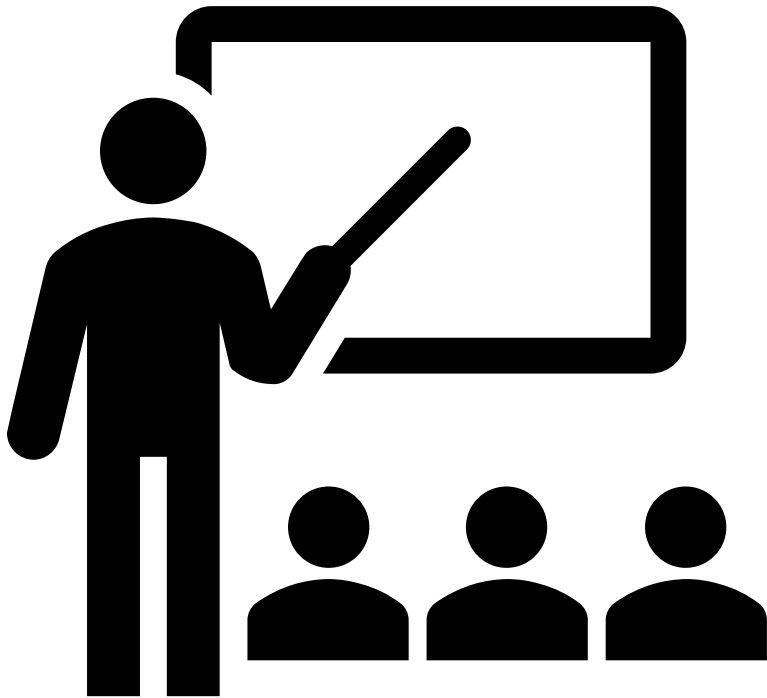


Bonus de
participation
😊

Objectifs pédagogiques

- 1** Maîtriser le cycle de vie des projets en Machine Learning (apprentissage automatique) et Deep Learning (apprentissage profond) : extraction, préparation, modélisation et évaluation
- 2** Implémenter des algorithmes simples puis avancés d'apprentissage automatique et profond pour des tâches supervisées et non supervisées
- 3** Évaluer les performances des modèles et optimiser les architectures pour répondre à des besoins spécifiques
- 4** Présenter des résultats sous forme de graphiques, de visuels, et de tableaux de bord interactifs et accessibles

Contenu du cours

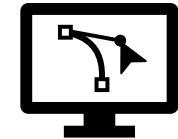


1



Fondamentaux du
Machine & Deep
Learning

2



Extraction, préparation,
visualisation et
communication des
données

3



Modélisation
supervisée et non-
supervisée en Machine
Learning

4



Modélisation
supervisée et avancée
en Deep Learning

I. Fondamentaux du Machine & Deep Learning



Comprendre les concepts clés et les bases théoriques du Machine & Deep Learning

1.1

Introduction au Machine & Deep Learning : concepts, historique, applications

1.2

Types d'apprentissages en Machine & Deep Learning

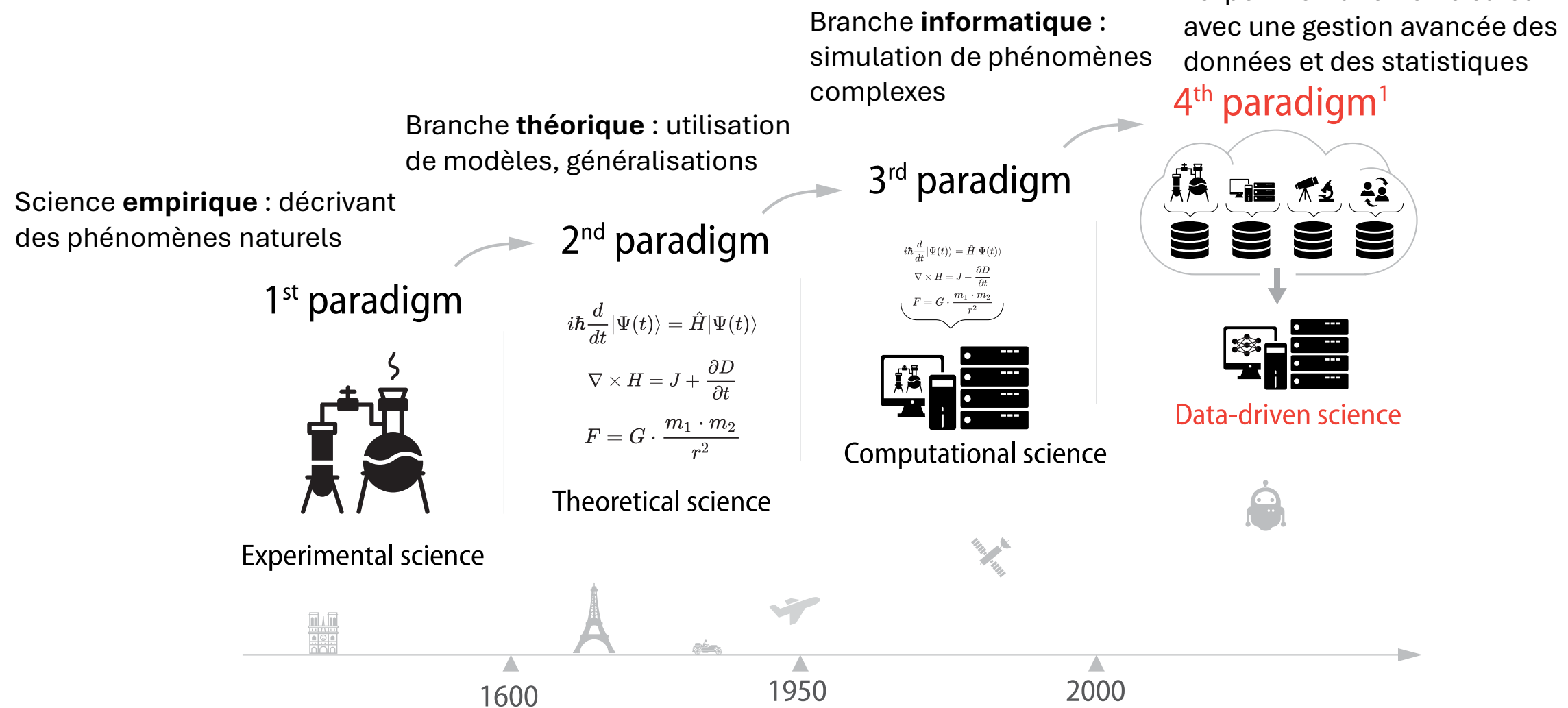
1.3

Principales difficultés liées au Machine & Deep Learning

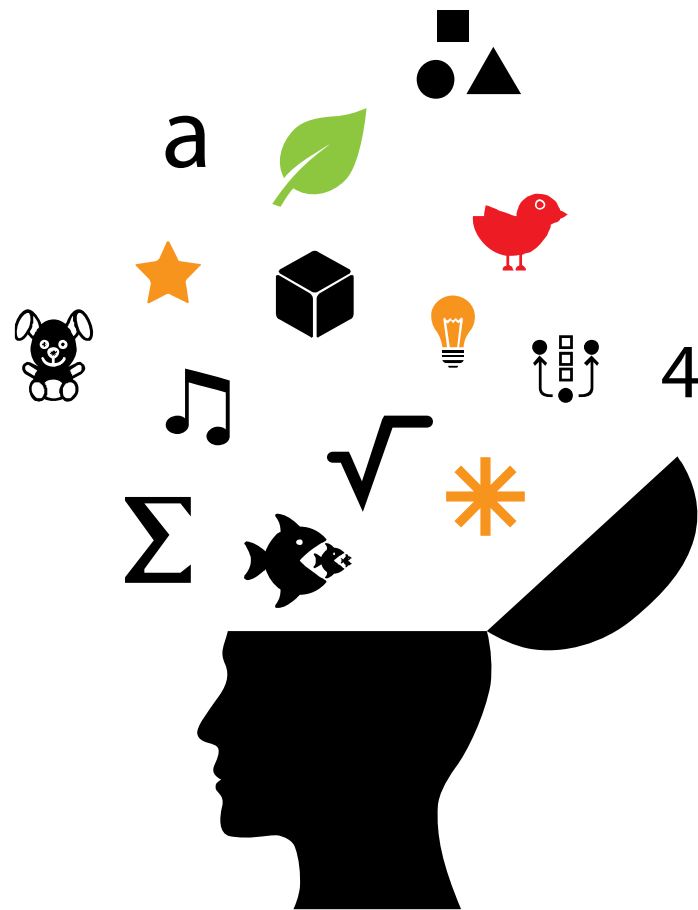
1.4

Présentation des frameworks : Scikit-learn, TensorFlow, PyTorch, Keras...

Science en évolution



¹J. Gray, Science in an exponential world



[INTELLIGENCE]

« Capacité de percevoir ou d'inférer l'information, et de la conserver comme une connaissance à appliquer à des comportements adaptatifs dans un environnement ou un contexte donné »

« Ensemble des fonctions mentales ayant pour objet la connaissance conceptuelle et rationnelle »*

* <https://www.larousse.fr>

La grande controverse

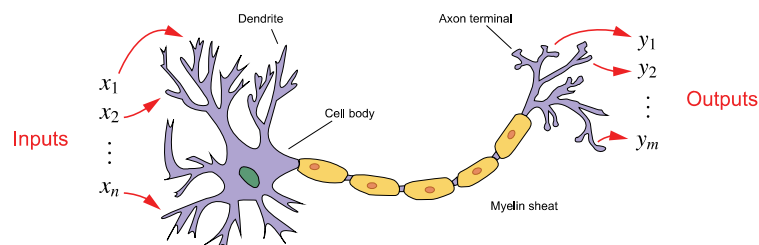
Modelling the brain :

« Penser s'apparente
à un calcul massivement parallèle de
fonctions élémentaires.

L'information est un **signal** avant
d'être un code »¹

Connexionniste

Modelling the brain
Modéliser le cerveau



Making a mind :

« Penser, c'est calculer des **symboles** qui
ont à la fois une réalité matérielle et une
valeur sémantique de représentation »¹

L'information est une donnée
symbolique de **haut niveau**.

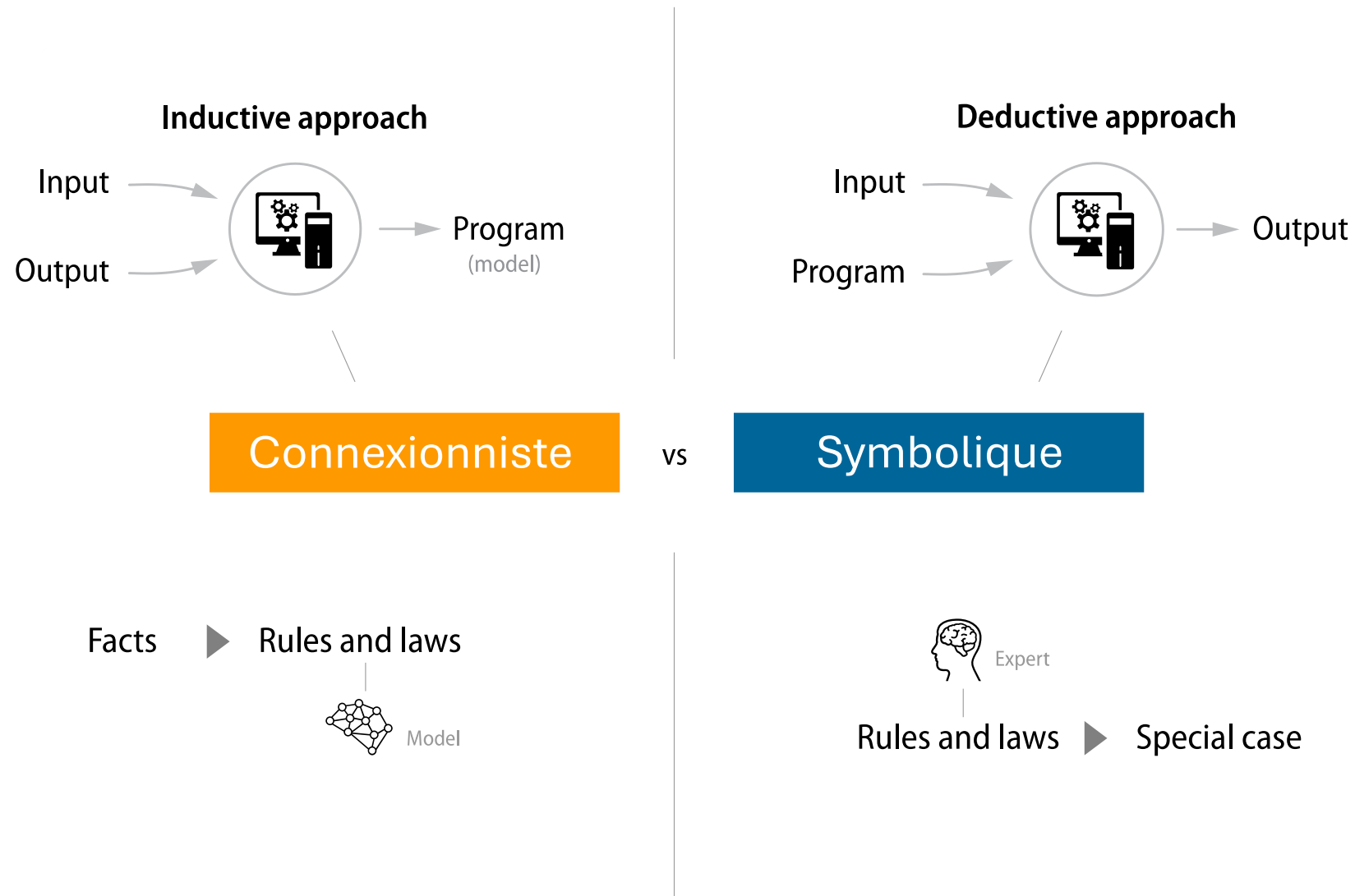
Symbolique

Making a mind
Forger une opinion

Tout [homme] est [mortel]
[Socrate] est un [homme]
Donc [Socrate] est [mortel]

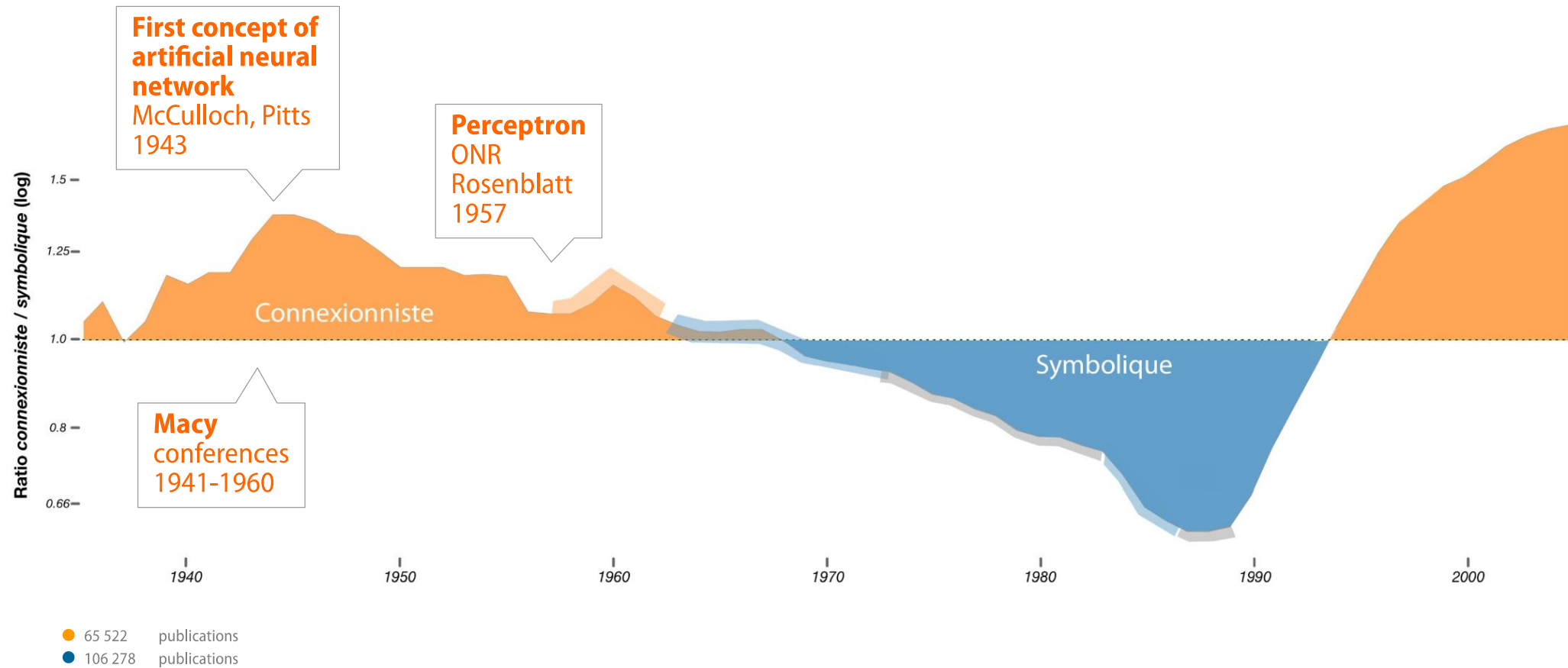
¹ D. Cardon, J.P. Cointet, A. Mazieres, 2018

La grande controverse



La grande controverse

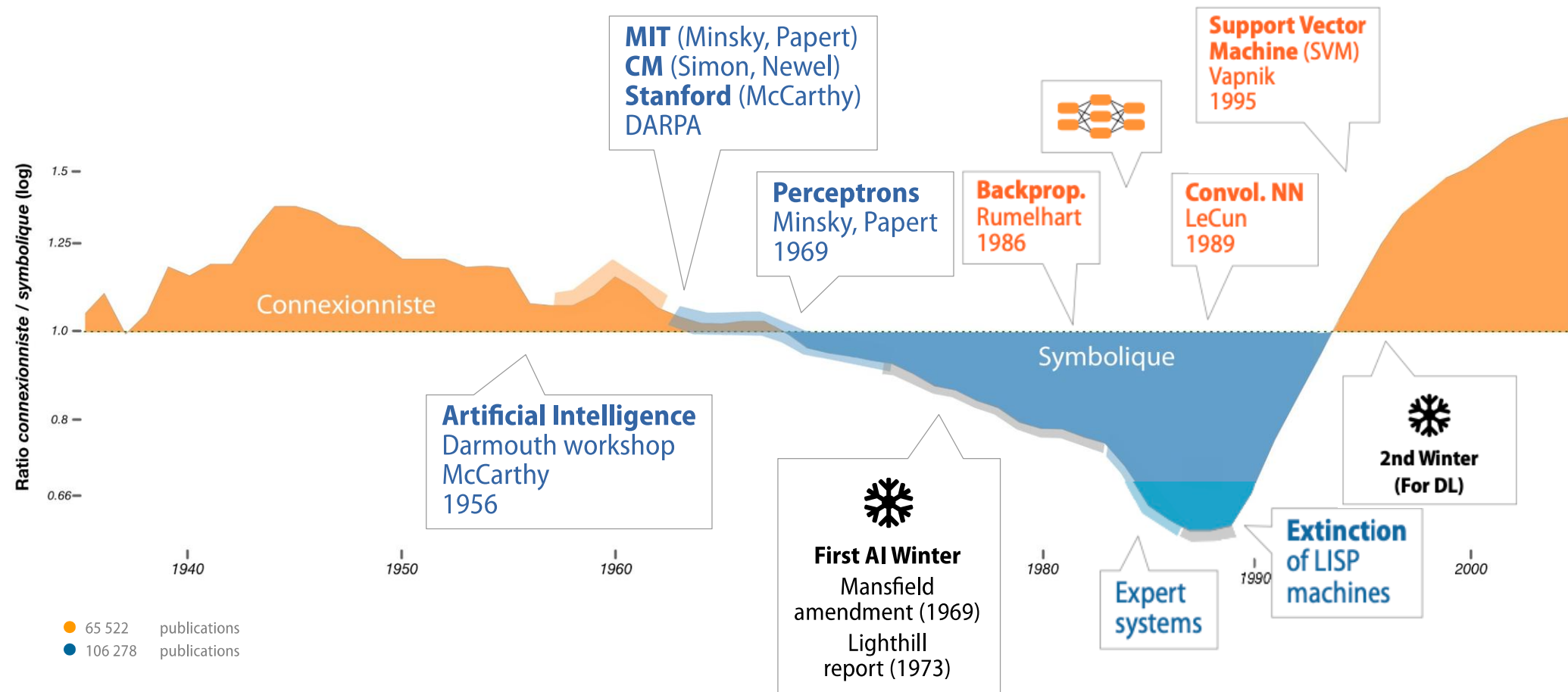
Evolution de l'influence académique des approches connexionnistes et symboliques¹



¹ D. Cardon, J.P. Cointet, A. Mazieres, 2018

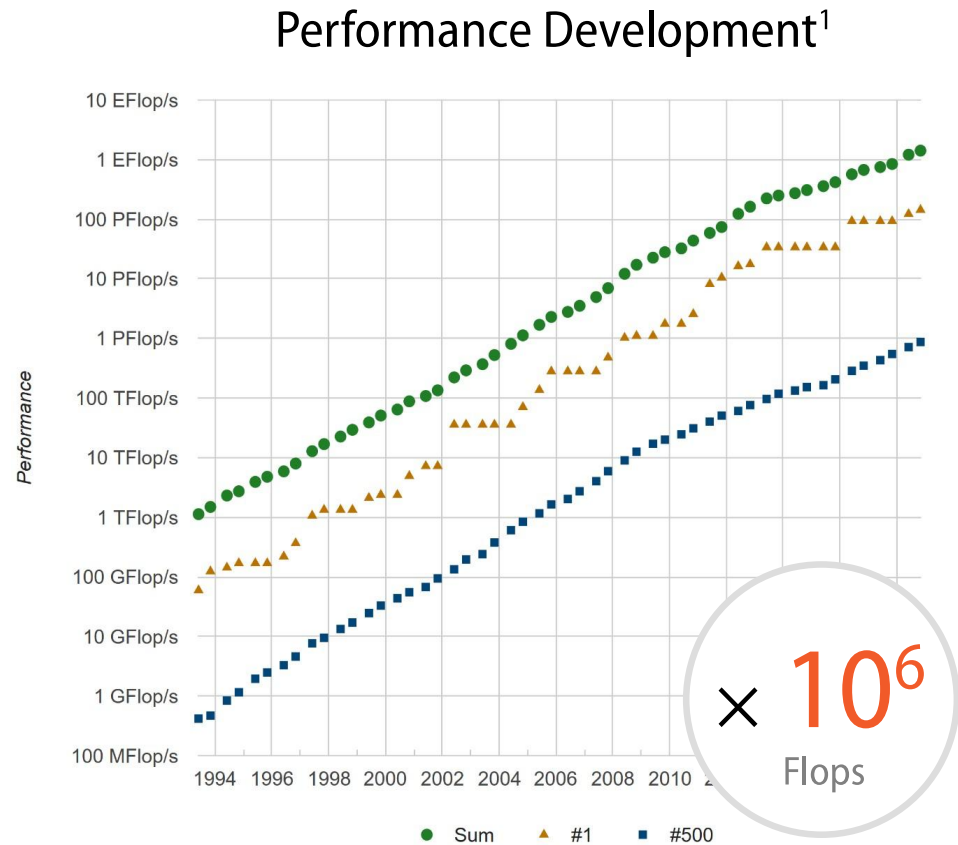
La grande controverse

Evolution de l'influence académique des approches connexionnistes et symboliques¹



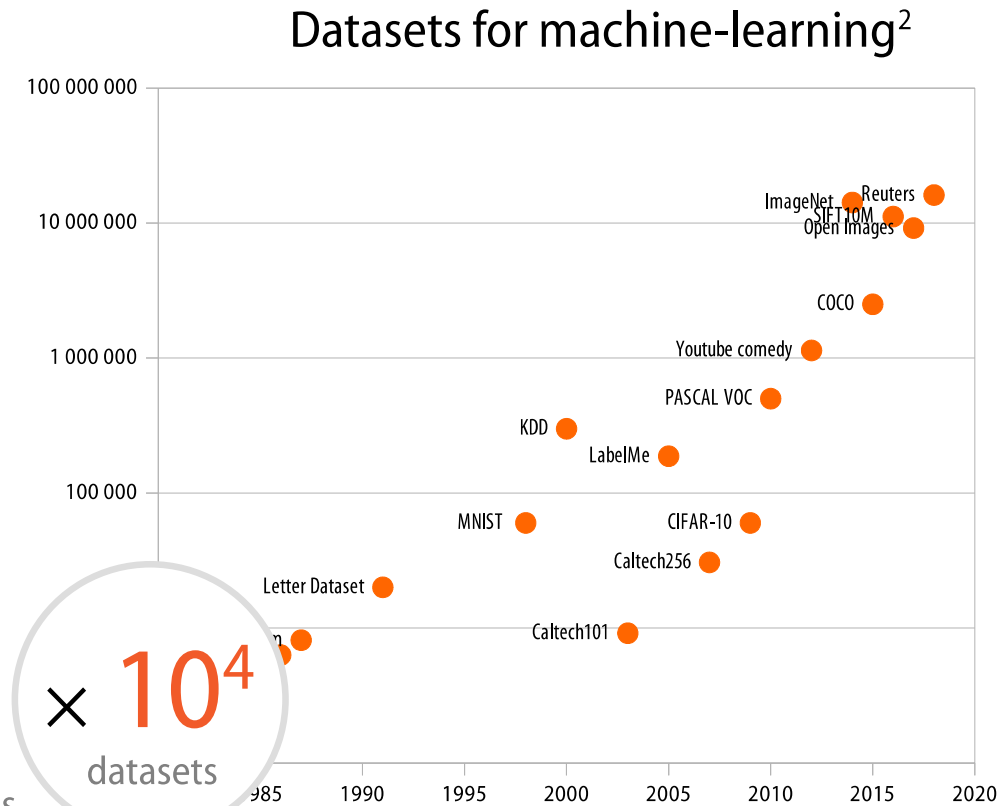
¹ D. Cardon, J.P. Cointet, A. Mazieres, 2018

La grande controverse



× 10^6
Flops

25 ans



× 10^4
datasets

Laboratoire
Cas particuliers



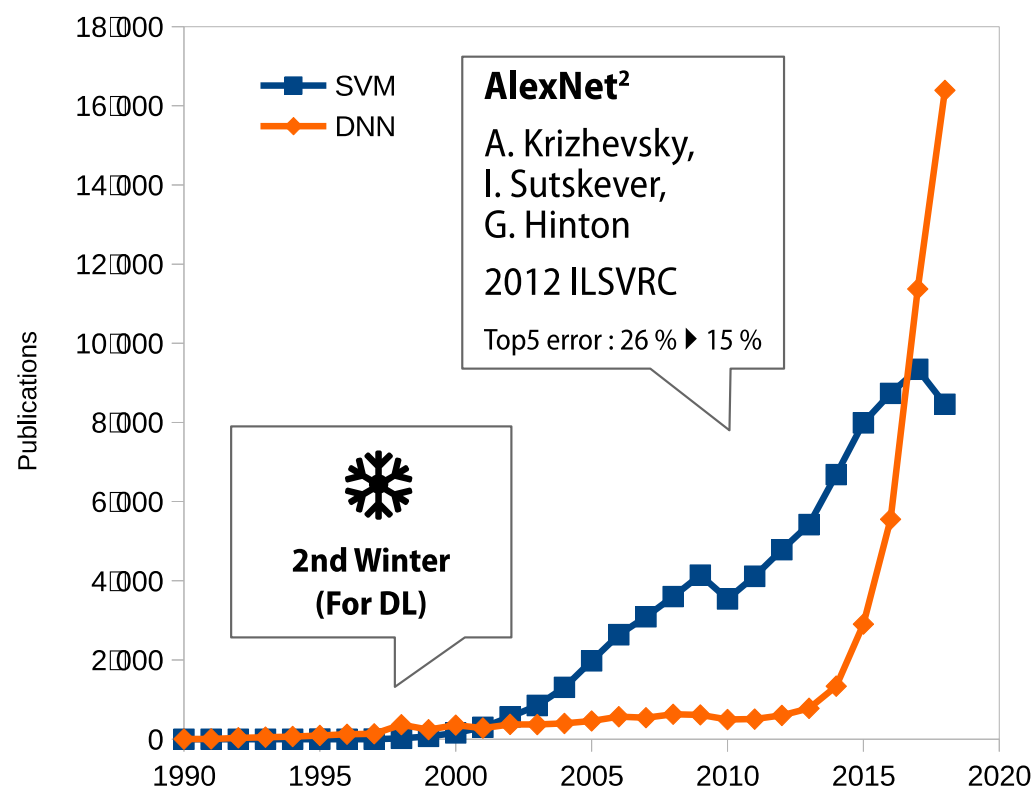
Monde réel
Utilisation générale

¹ TOP500 List

² Wikipedia

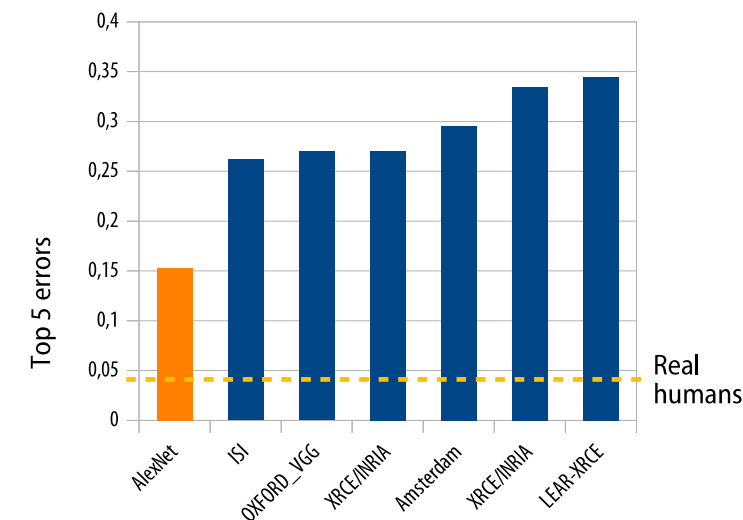
La grande controverse

Publications SVM vs DNN¹



DNN

Images classification
Top 5 error at ILSVRC 2012^{3,4}



Sans garantie mathématique,
les DNN se sont révélés plus
efficaces face à la complexité
du monde réel !

¹ Web of Science

² AlexNet

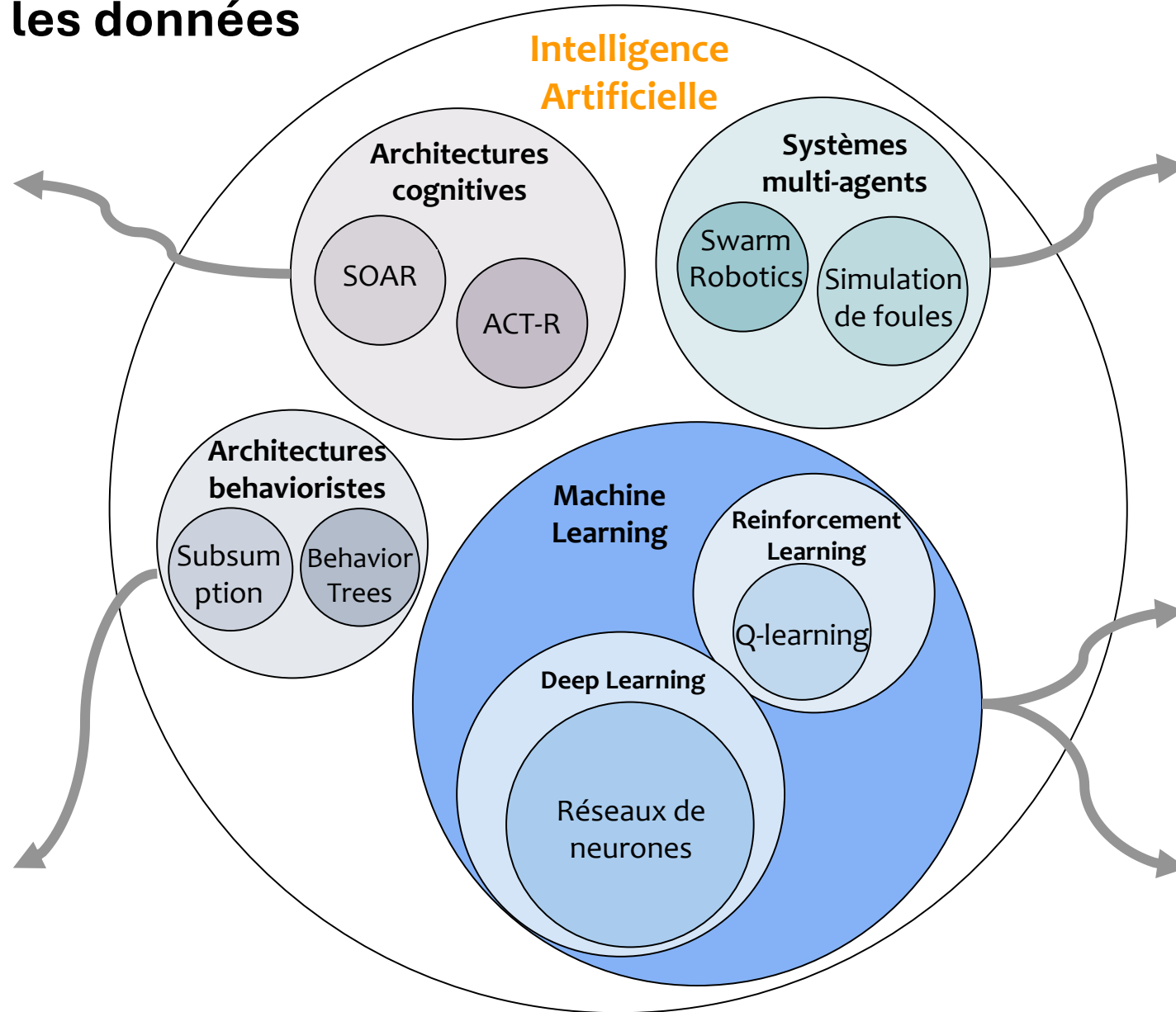
³ ImageNet Large Scale Visual Recognition [ILSVRC]

⁴ Evolution similaire dans le traitement du langage naturel, la traduction, les jeux de société, etc.
Voir : DeepL.com, AlphaGo, AlphaZero,...

Science basée sur les données

Modèles informatiques qui imitent le fonctionnement de l'esprit humain pour simuler des processus comme la mémoire, la perception ou le raisonnement.

Méthodes de contrôle basées sur des comportements simples et réactifs, souvent utilisées en robotique.



Ensemble de programmes autonomes (agents) qui interagissent entre eux pour accomplir une tâche collective.

Technique où un agent apprend à prendre des décisions en recevant des récompenses ou des pénalités selon ses actions.

Modèles inspirés du cerveau humain, utilisés pour apprendre à partir de grandes quantités de données.

Science basée sur les données

Qu'est-ce que le « Machine Learning » (ou l'apprentissage automatique) ?

C'est la science de programmer les ordinateurs de sorte qu'ils puissent apprendre à partir de données.



1

Comment le Machine Learning permet-il d'extraire de l'information utile à partir de données brutes pour améliorer la prise de décision ?

2

En quoi le Machine Learning peut-il automatiser la prise de décision à partir de données observées ?

3

Comment un système basé sur le Machine Learning peut-il s'adapter à un environnement en constante évolution ?



Apprentissage automatique VS Apprentissage statistique

Critères	Apprentissage statistique	Apprentissage automatique
Dimension des données	Faible dimension (souvent < 100 variables)	Très grande dimension (jusqu'à des dizaines de milliers de variables)
Qualité des données	Données très bruitées, souvent issues d'enquêtes ou de mesures imparfaites	Données plus propres, souvent issues de capteurs ou de systèmes numériques
Structure des données	Peu de structure apparente, modélisation par des modèles paramétriques simples	Structure complexe et souvent non linéaire, difficile à modéliser explicitement
Objectif principal	Séparer le signal du bruit	Capturer et exploiter la structure cachée des données
Méthodes utilisées	Régressions linéaires, modèles probabilistes, tests statistiques	Réseaux de neurones, forêts aléatoires, SVM, boosting, etc.
Validation théorique	Preuves mathématiques (convergence, biais/variance, etc.)	Validation empirique (expériences, benchmarks, cas d'usage)
Approche	Approche explicative : comprendre les relations entre variables	Approche prédictive : optimiser les performances sur des tâches spécifiques

Quelques applications de l'apprentissage automatique



Filtre anti-spam



Tri postal automatique (détection de chiffres et de lettres manuscrits)



Analyse des profils des clients pour proposer des produits : Stratégie Marketing



Programmes qui gagnent face à des champions du monde de jeux (1997 : DeepBlue, 2017: Alpha GO)

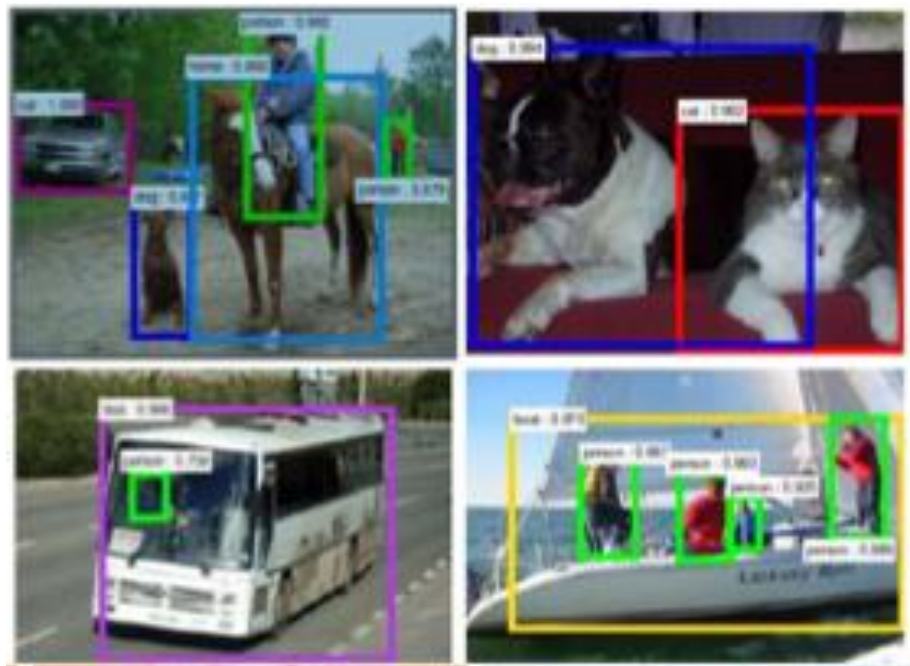


Appareil photo avec détection de visage
→ filtres sur les visages



Chat bots : generation de texte, d'images, de sons, etc.

Quelques applications de l'apprentissage automatique



Détection et reconnaissance d'objets dans des images



Homme en t-shirt noir qui joue de la guitar



Un ouvrier du bâtiment portant un gilet de sécurité orange travaille



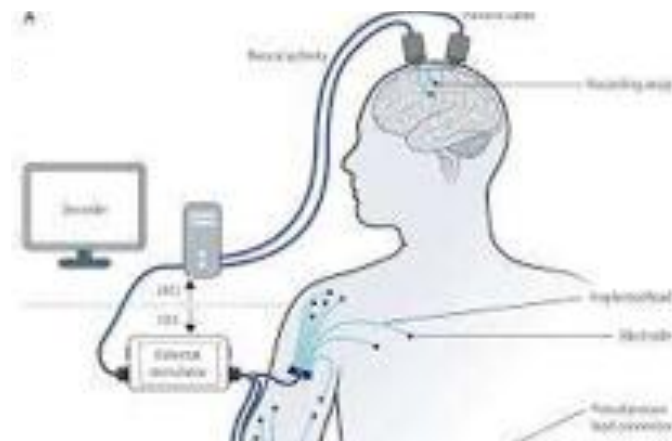
Deux jeunes filles jouent avec un jouet LEGO



Un garçon fait un salto arrière en wakeboard

Reconnaissance d'images et description de contenu

Quelques applications de l'apprentissage automatique



Contrôle des membres par la pensées pour des paralytiques (déchiffrement des ondes cérébrales)

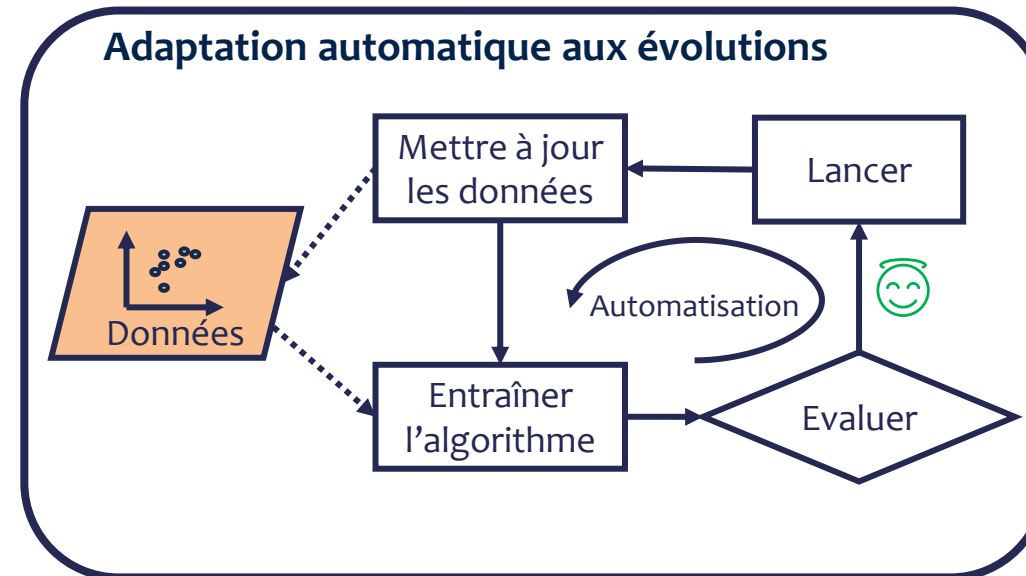
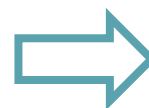
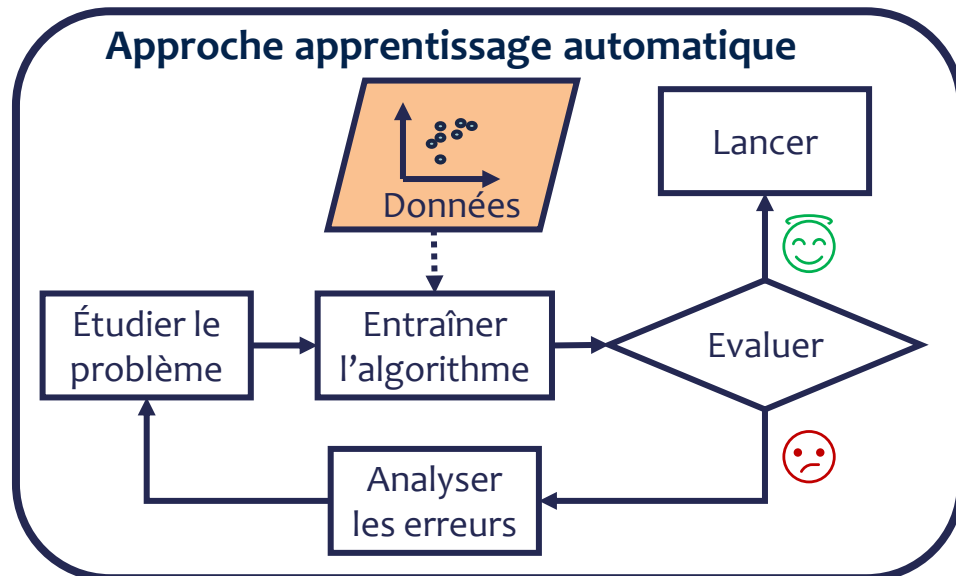
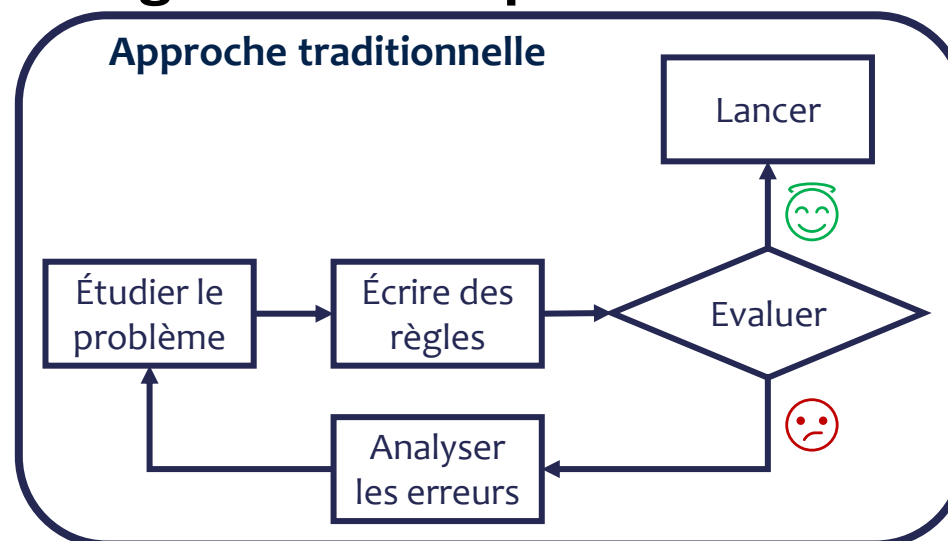


Détection et caractérisation de tumeurs

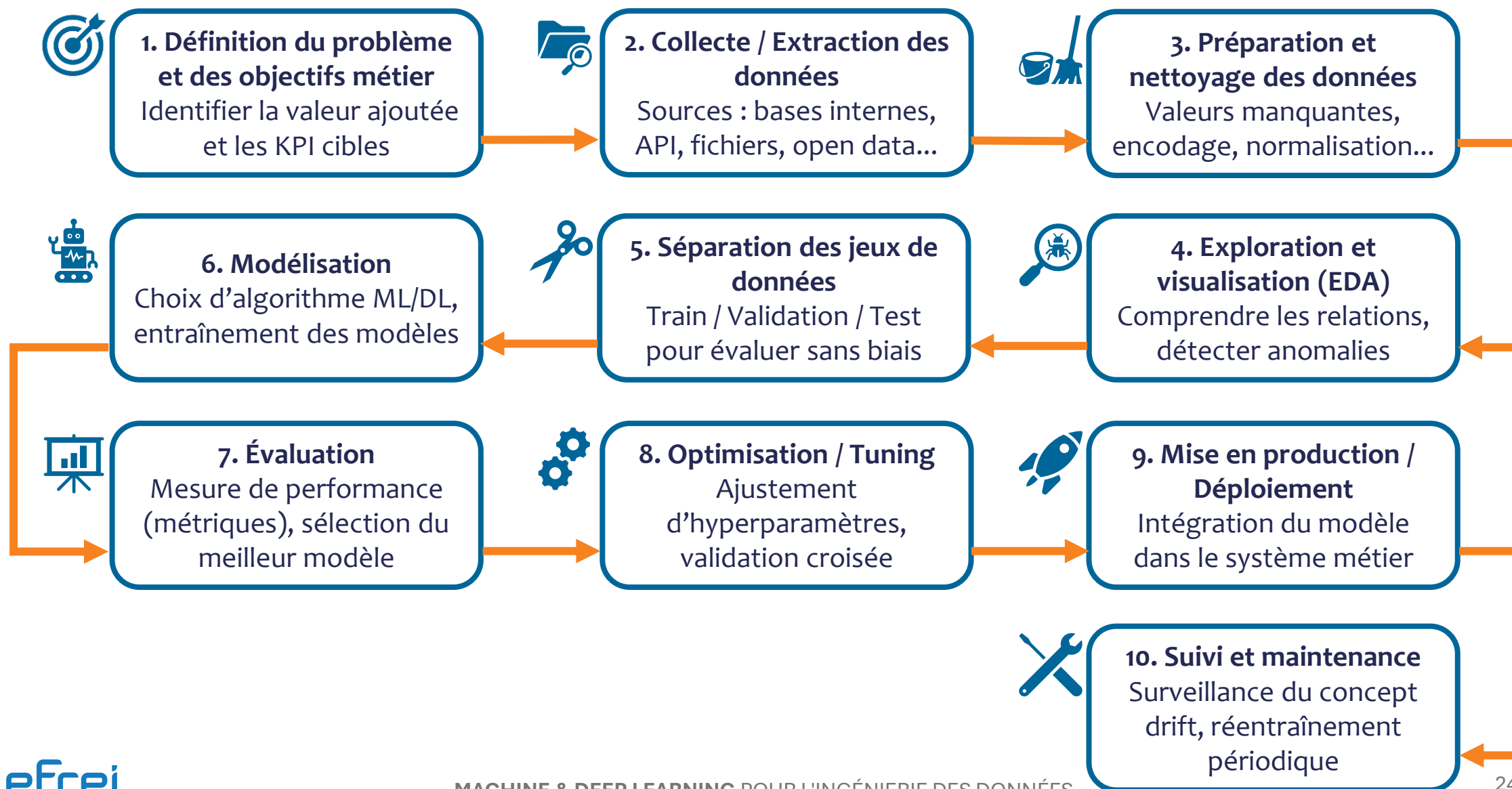


Véhicules autonomes : voitures, avions, trains, etc.

Pourquoi utiliser l'apprentissage automatique ?



Cycle de vie d'un projet de Machine Learning



I. Fondamentaux du Machine & Deep Learning



Comprendre les concepts clés et les bases théoriques du Machine & Deep Learning

1.1

Introduction au Machine & Deep Learning : concepts, historique, applications

1.2

Types d'apprentissages en Machine & Deep Learning

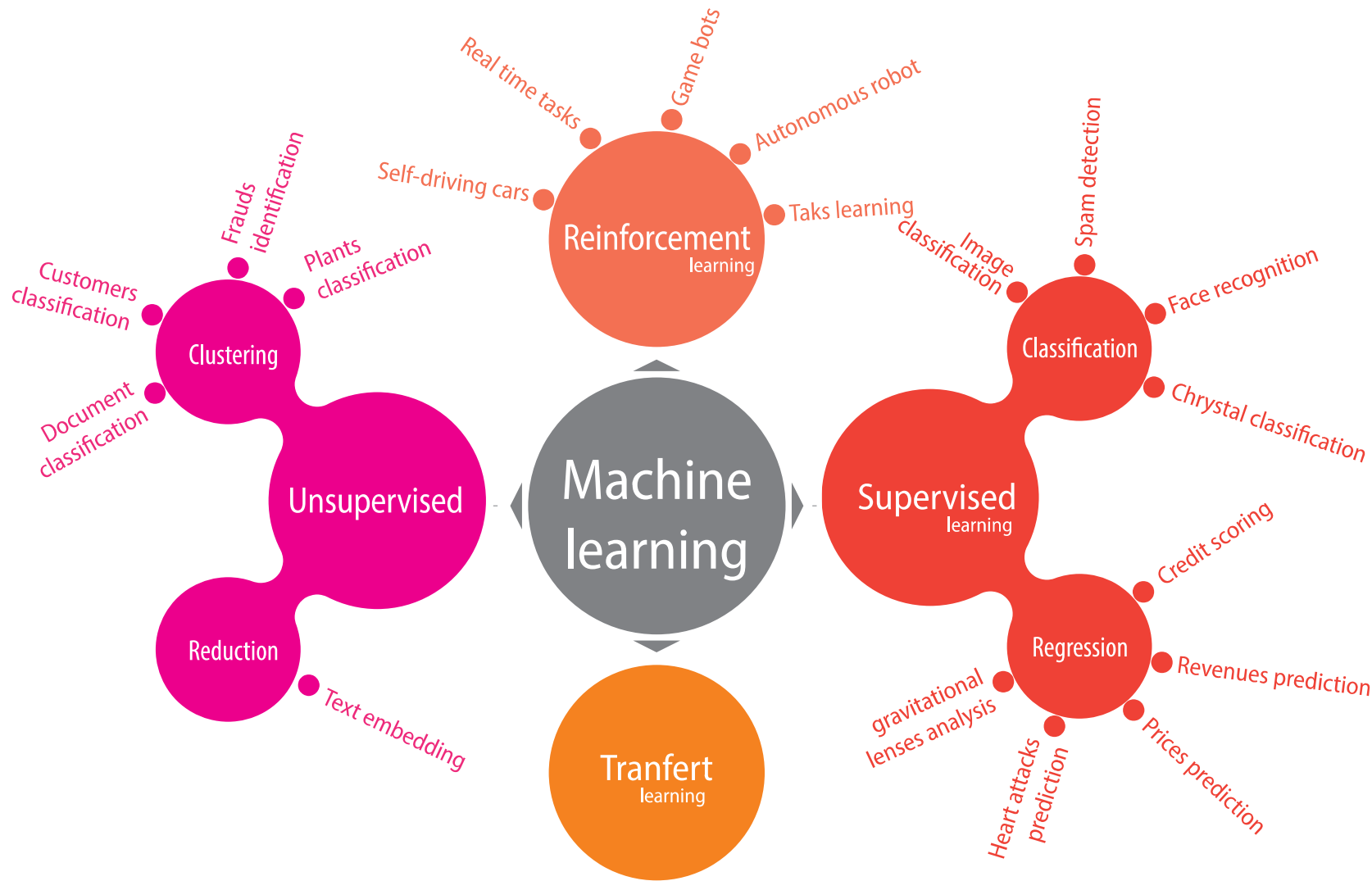
1.3

Principales difficultés liées au Machine & Deep Learning

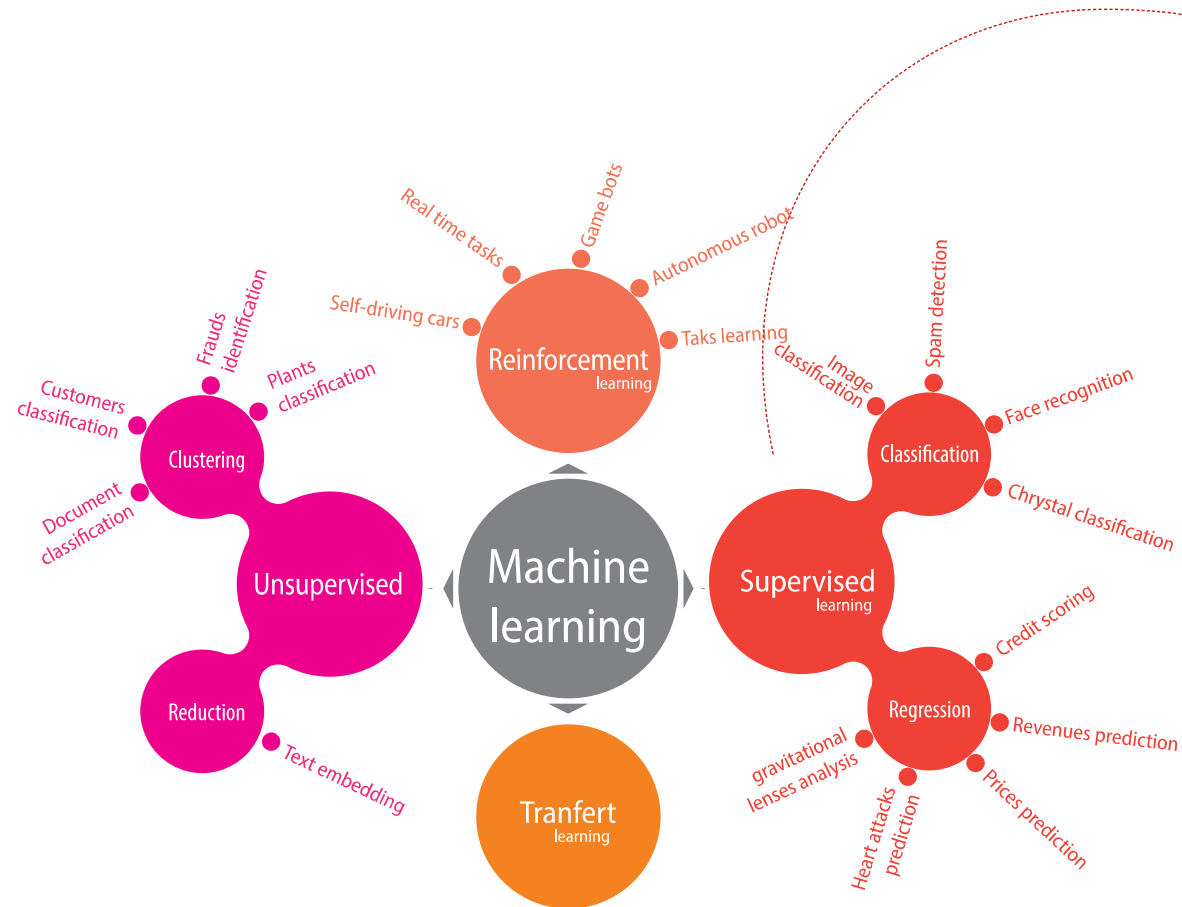
1.4

Présentation des frameworks : Scikit-learn, TensorFlow, PyTorch, Keras...

Différents types d'apprentissage



Apprentissage supervisé



Apprendre
des **exemples**

Apprentissage supervisé : apprendre des exemples

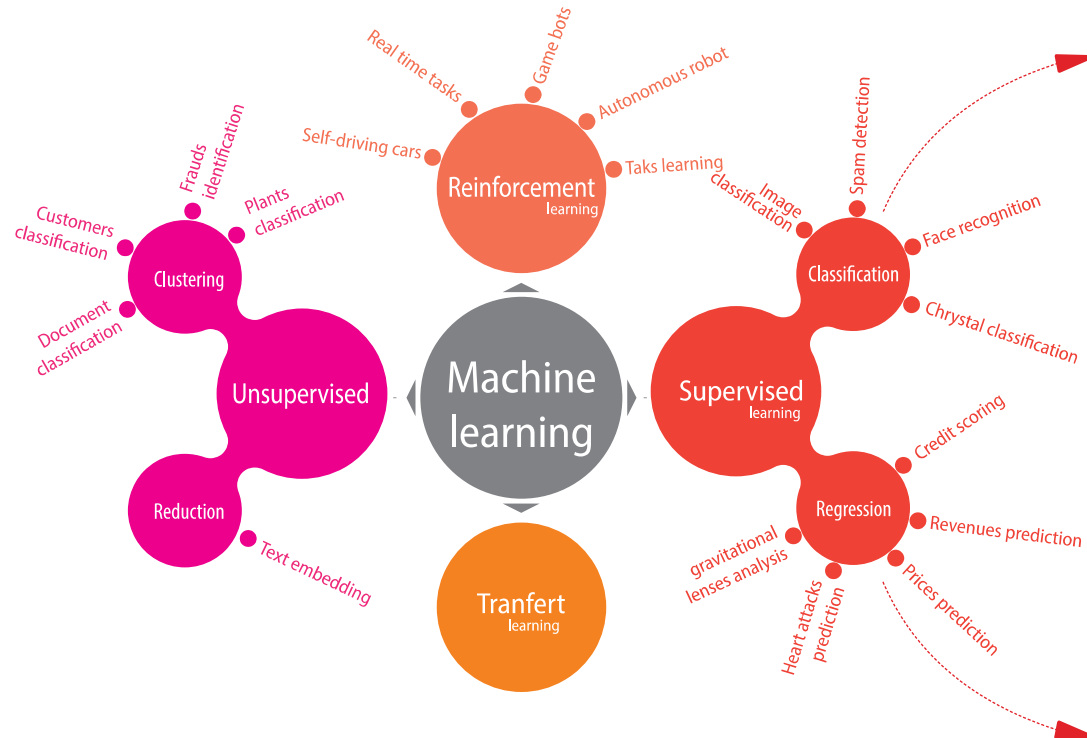
Comment fonctionne l'apprentissage supervisé ?

Apprentissage supervisé = un apprentissage qui est guidé, dirigé, encadré par des exemples préalablement fournis.



- Un algorithme va apprendre à faire une tâche en étudiant plusieurs **exemples** de cette tâche.
- Les exemples d'apprentissage/entrainement comportent des solutions désirées appelées **étiquettes** (labels).
- Pour cette tâche, l'algorithme peut apprendre une **relation** $f: x \rightarrow y$ en **analysant** les différents exemples d'associations x, y présents dans les exemples d'apprentissage.

Apprentissage supervisé : apprendre des exemples



Classification :

Prédire les informations qualitatives



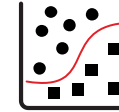
Ceci est un chat



Ceci est un lapin



Dis-moi,
qu'est-ce que
c'est ?



Régression :

Prédire les informations quantitatives



150 K€



400 K€



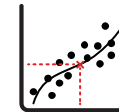
120 K€



100 K€



Dis-moi, quel
est le prix ?



Apprentissage supervisé : apprendre des exemples



D'où viennent les étiquettes ?

- **Des utilisateurs eux-mêmes** → Par exemple, sur les réseaux sociaux, les photos ou publications sont souvent accompagnées de **hashtags** ou de **légendes** qui servent implicitement d'étiquettes.
- **De l'annotation humaine** → Des **travailleurs humains** (internes ou via des plateformes d'annotation) nettoient, classent ou qualifient les données manuellement. Ce processus est appelé **annotation de données** et permet de créer des jeux de données supervisés fiables.
- **Des données elles-mêmes (étiquettes implicites)** → Dans certains cas, l'étiquette découle naturellement de la donnée observée. Exemple : dans un jeu de données sur l'immobilier, le **prix de vente** d'un bien au fil du temps peut servir d'étiquette pour un modèle de prédiction de prix.

Apprentissage supervisé : apprendre des exemples



Quelles conditions pour réussir un apprentissage supervisé?

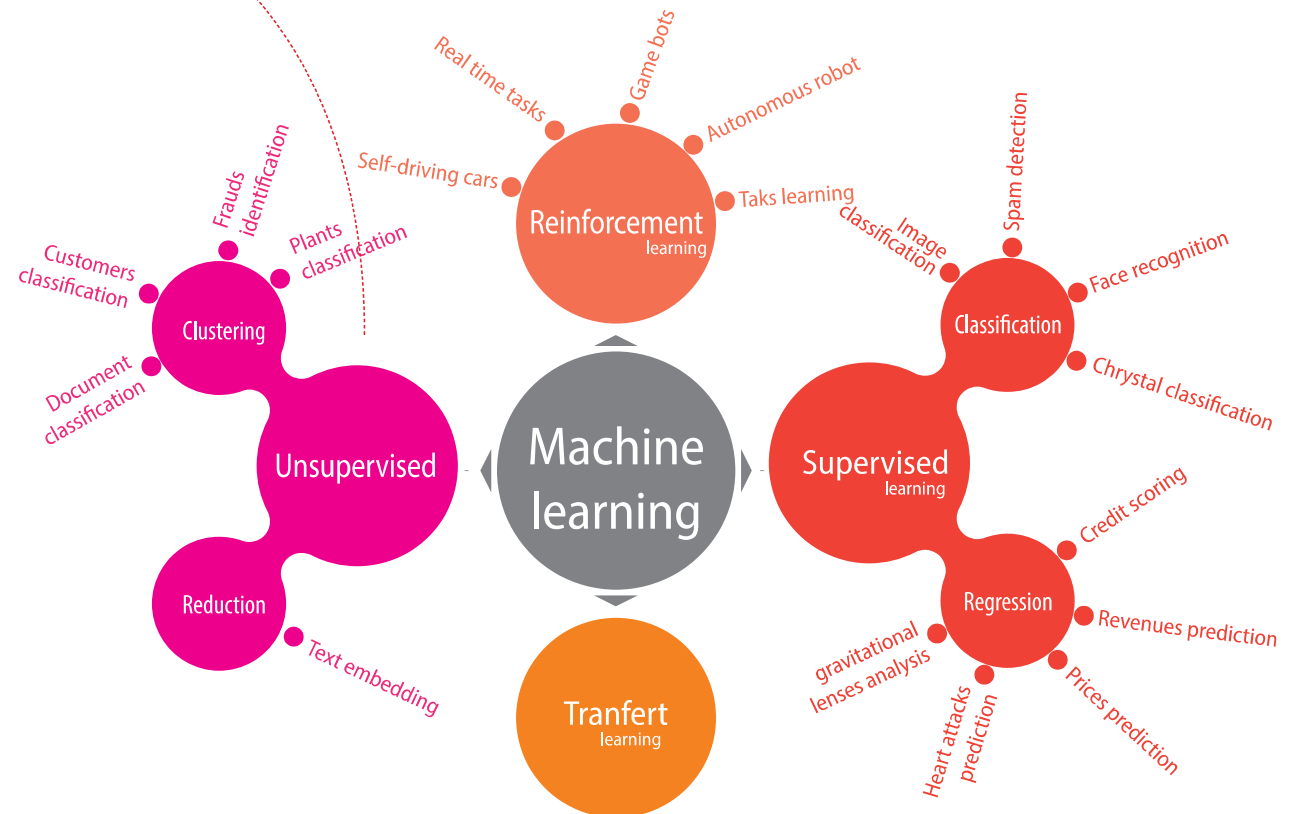
- **Disposer d'un volume de données suffisant et représentatif** → Un modèle supervisé apprend à partir d'exemples : plus les **données d'entraînement** sont nombreuses, variées et équilibrées, meilleure sera sa capacité de généralisation.
- **S'assurer que les nouvelles situations ressemblent à celles vues pendant l'entraînement** → Le modèle ne peut bien prédire que sur des **données similaires** à celles du jeu d'apprentissage. En cas de contexte trop différent (*dérive de concept*, nouveaux comportements, évolution du marché...), les performances chutent.
- **Veiller à la qualité et à la pertinence des données d'apprentissage** → Les données doivent être **spécifiques au problème à résoudre, propres** (sans bruit ni erreurs) et **bien étiquetées**. Une mauvaise qualité d'étiquettes ou un biais dans l'échantillon compromettra les résultats.

Chihuahua ou muffin ?



Apprentissage non-supervisé

Apprendre des données en autonomie



Apprentissage non-supervisé : apprendre des données en autonomie

Comment fonctionne l'apprentissage non-supervisé ?

Apprentissage non supervisé = un apprentissage sans guide ni étiquette.

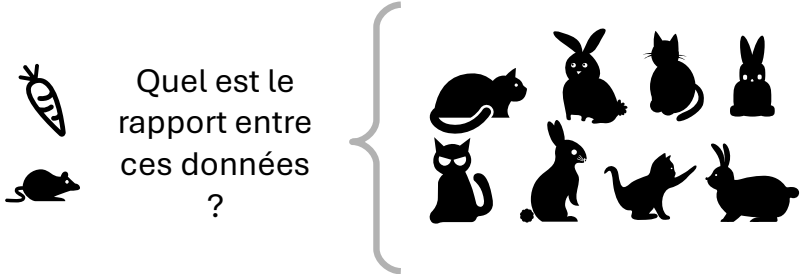


- L'algorithme n'a pas accès aux réponses attendues → il explore les données pour en **découvrir la structure cachée** ou les **relations naturelles** entre les exemples.
- L'algorithme apprend uniquement à partir des **caractéristiques (x)** des données, sans connaître les **étiquettes (y)**. Il cherche à **regrouper** les observations similaires, à **détecter des schémas**, ou à **réduire la complexité** des données.
- Pour cette tâche, l'algorithme apprend une fonction $f: x \rightarrow x'$, où x' représente une représentation simplifiée ou structurée des données (des groupes, des clusters ou des axes principaux).

Apprentissage non-supervisé : apprendre des données en autonomie

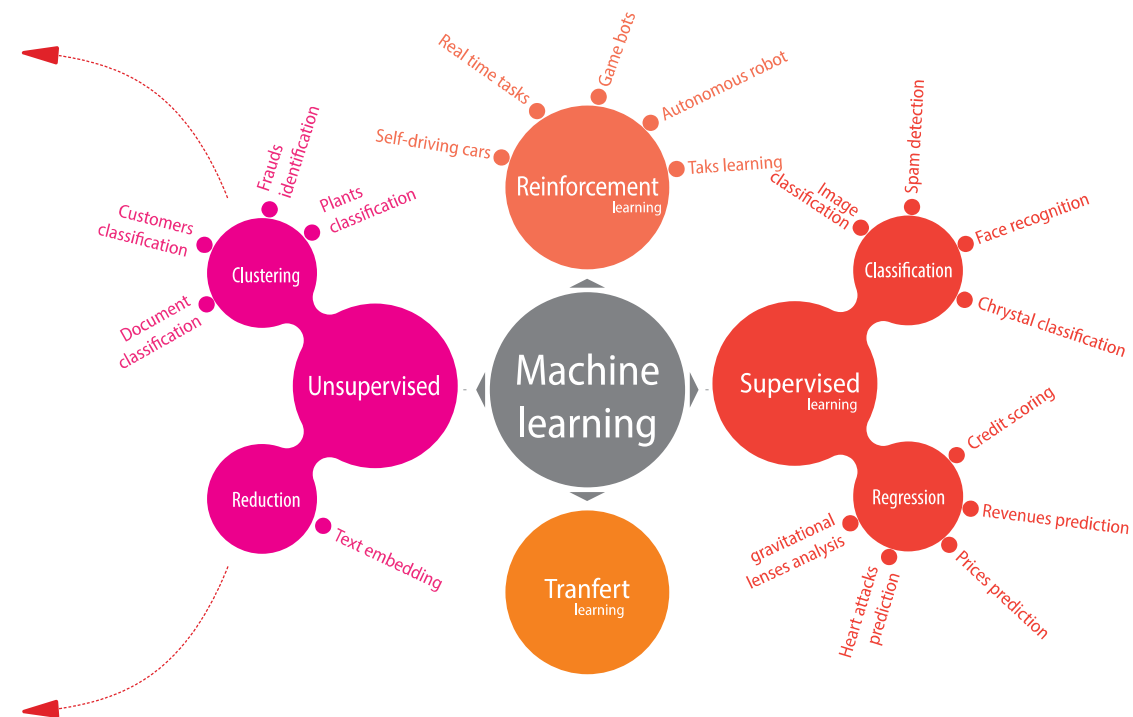
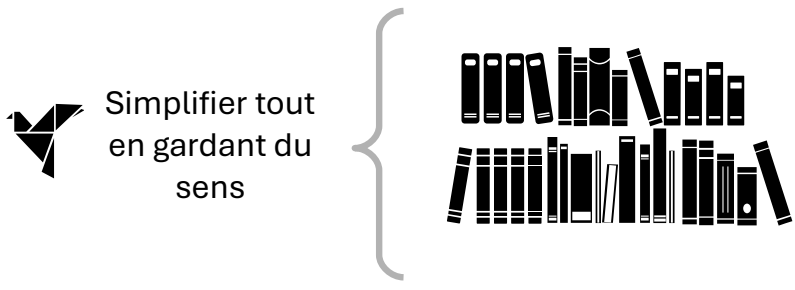
Clustering :
Trouver des relations communes

Quel est le rapport entre ces données ?



Reduction :
Réduire le nombre de dimensions

Simplifier tout en gardant du sens



Apprentissage non-supervisé : apprendre des données en autonomie



Des exemples ?

Visiteurs d'un blog

Effectuer un partitionnement (clustering) pour tenter de détecter des groupes de visiteurs similaires.

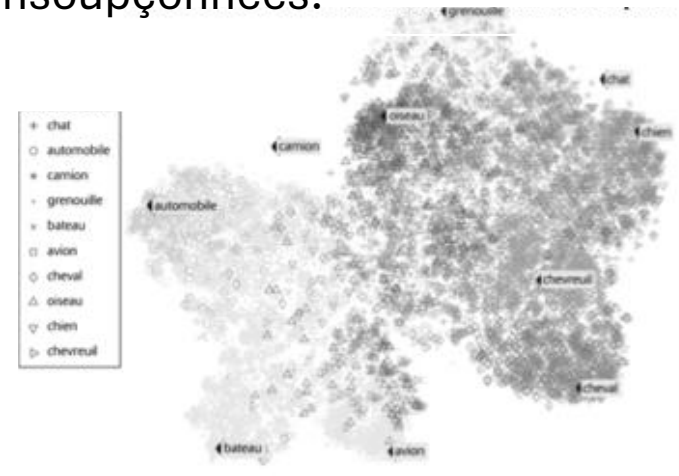
- X% des visiteurs sont des hommes adorant les bandes dessinées et lisant le blog le soir
- Y% des jeunes passionnés de sciences fiction qui le consultent le blog week-end



Algorithmes de visualisation

Fournir une représentation en 2D ou 3D sous forme d'un nuage de pts d'un grand nombre de données complexes non étiquetées

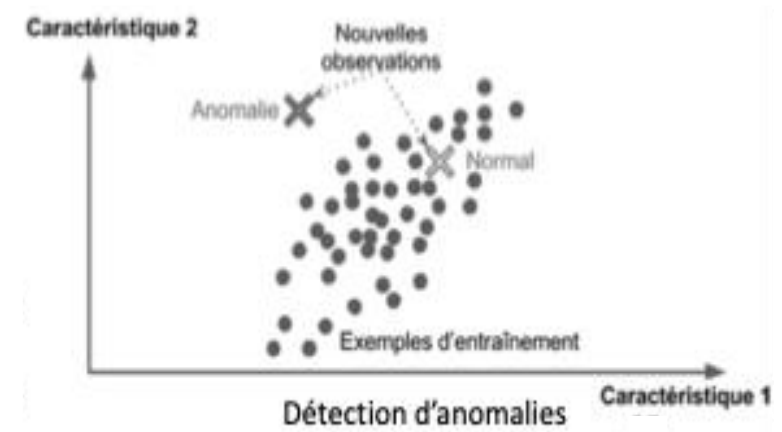
But: Comprendre comment les données sont organisées pour identifier des caractéristiques insoupçonnées.



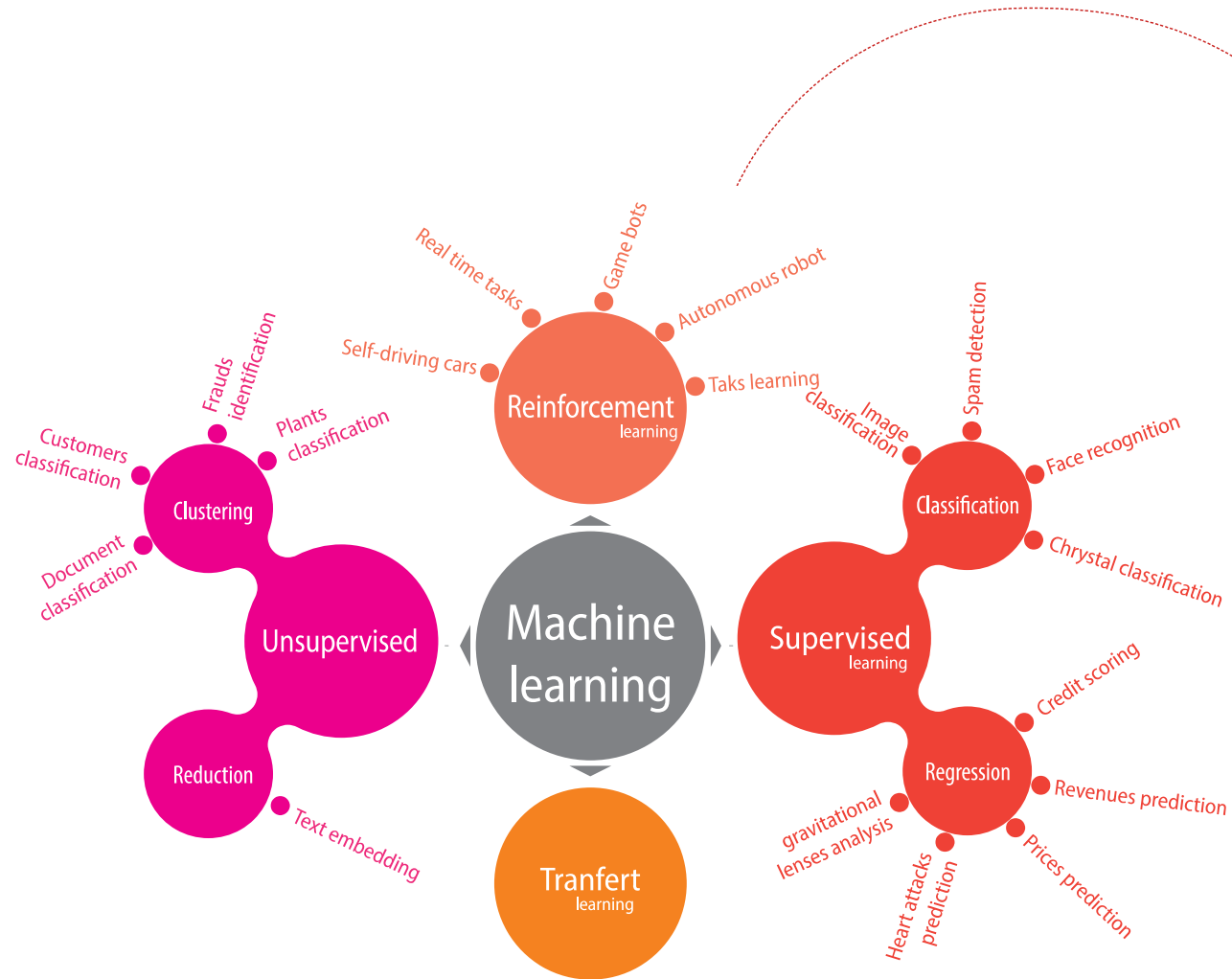
Détection d'anomalies

Durant l'apprentissage: L'algorithme reçoit des exemples normales (pour apprendre à les connaître)

Durant le test : Lorsqu'il recevra un nouvel exemple, il pourra déterminer si elle est normale ou si c'est une anomalie.



Apprentissage par renforcement



Apprendre par la récompense

Apprentissage par renforcement : apprendre par la récompense

Comment fonctionne l'apprentissage par renforcement ?

Apprentissage par renforcement = un apprentissage basé sur l'expérience et la récompense.



- L'algorithme n'apprend pas à partir d'exemples, mais en **interagissant avec un environnement**.
- À chaque action effectuée, il reçoit une **récompense** (positive ou négative) qui lui indique si son comportement est adapté. En répétant ces interactions, il apprend progressivement **quelle stratégie ou suite d'actions maximise la récompense totale**.
- Pour une tâche, l'algorithme cherche à apprendre une **politique optimale** $\pi : \pi(s) \rightarrow a$, c'est-à-dire une fonction qui associe à chaque **état** s de l'environnement l'**action** a la plus avantageuse à long terme.

Apprentissage par renforcement : apprendre par la récompense

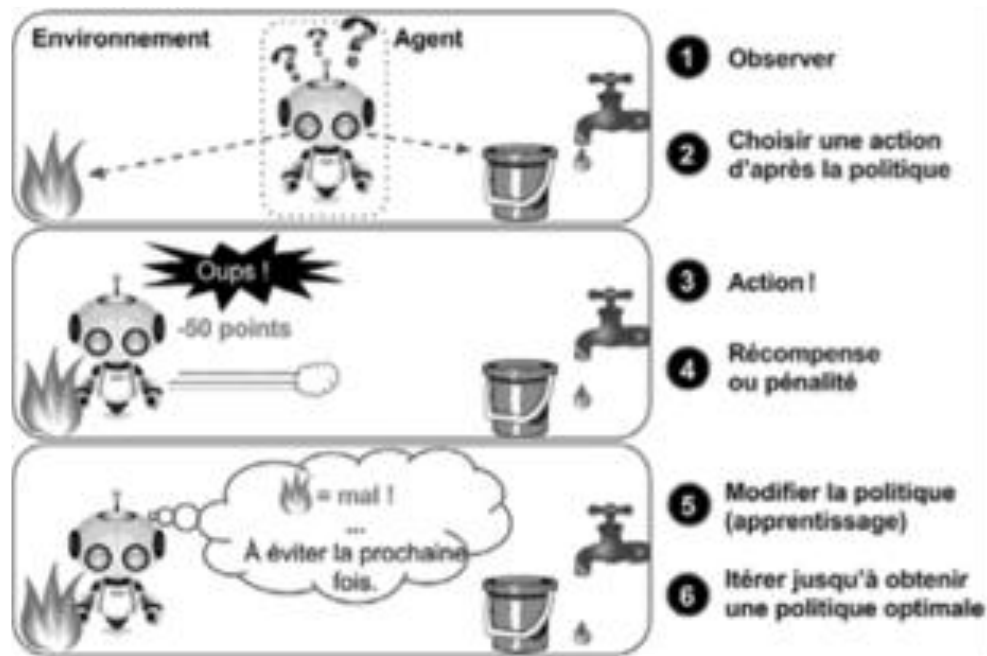


Quel est le principe de base ?

- Le système d'apprentissage est appelé « agent » observe son environnement, sélectionne et accomplit des actions et obtient en retour une récompense (ou une pénalité)
- Il finit par apprendre par lui-même la meilleure stratégie pour obtenir le maximum de récompenses.
- La stratégie (politique) définit quelle action choisir face à une situation donnée.

Quelques exemples ?

- Des robots (autonomes) l'utilisent pour apprendre à bouger : Boston Dynamics, Unitree Robotics, Optimus,... [Démon vidéo 1](#), [Démon vidéo 2](#)
- Apprendre des jeux : AlphaGo de DeepMind l'a utilisé pour battre le meilleur joueur de go en faisant la une des journaux en mai 2017.
- Pilotage de véhicules intelligents (voitures, avions, trains,...)



Apprentissage par renforcement : apprendre par la récompense



Cas d'étude : Super Mario Bros

Objectif :

Maximiser la somme des récompenses reçues au fil du temps.

- **But du jeu** : atteindre la fin du niveau en récoltant des pièces
- **Résultats négatifs** : tomber dans un trou, se faire toucher par un ennemi, etc.
- **Actions possibles** : aller à droite/gauche, sauter, s'accroupir, tirer, etc.



Concepts clés :

- **Agent** : celui qui agit (ex. Mario)
- **Environnement** : le monde dans lequel il évolue (le jeu Super Mario Bros)
- **État (State)** : la situation actuelle de l'agent (position, obstacles visibles, etc.)
- **Action** : choix possible à chaque étape (avancer, sauter, tirer...)
- **Récompense** : signal indiquant le résultat d'une action (gain ou perte de points)
- **Politique (Policy)** : stratégie que l'agent apprend pour choisir la meilleure action selon l'état

Apprentissage par renforcement : apprendre par la récompense



Cas d'étude : Super Mario Bros

Cycle d'apprentissage par renforcement :

1. **Exploration** : Mario tente une action aléatoire (ex. se déplacer à droite).
2. **Observation** : Il observe le résultat : progression ou mort par un ennemi.
3. **Récompense** :
 - Action bénéfique → récompense positive (avancer, collecter pièces)
 - Action néfaste → récompense négative (mort, perte de points)
4. **Mise à jour de la politique** : Le système renforce les actions positives et évite les négatives.
5. **Itération** : En répétant ces étapes des **milliers de fois**, Mario apprend à **optimiser ses décisions**.

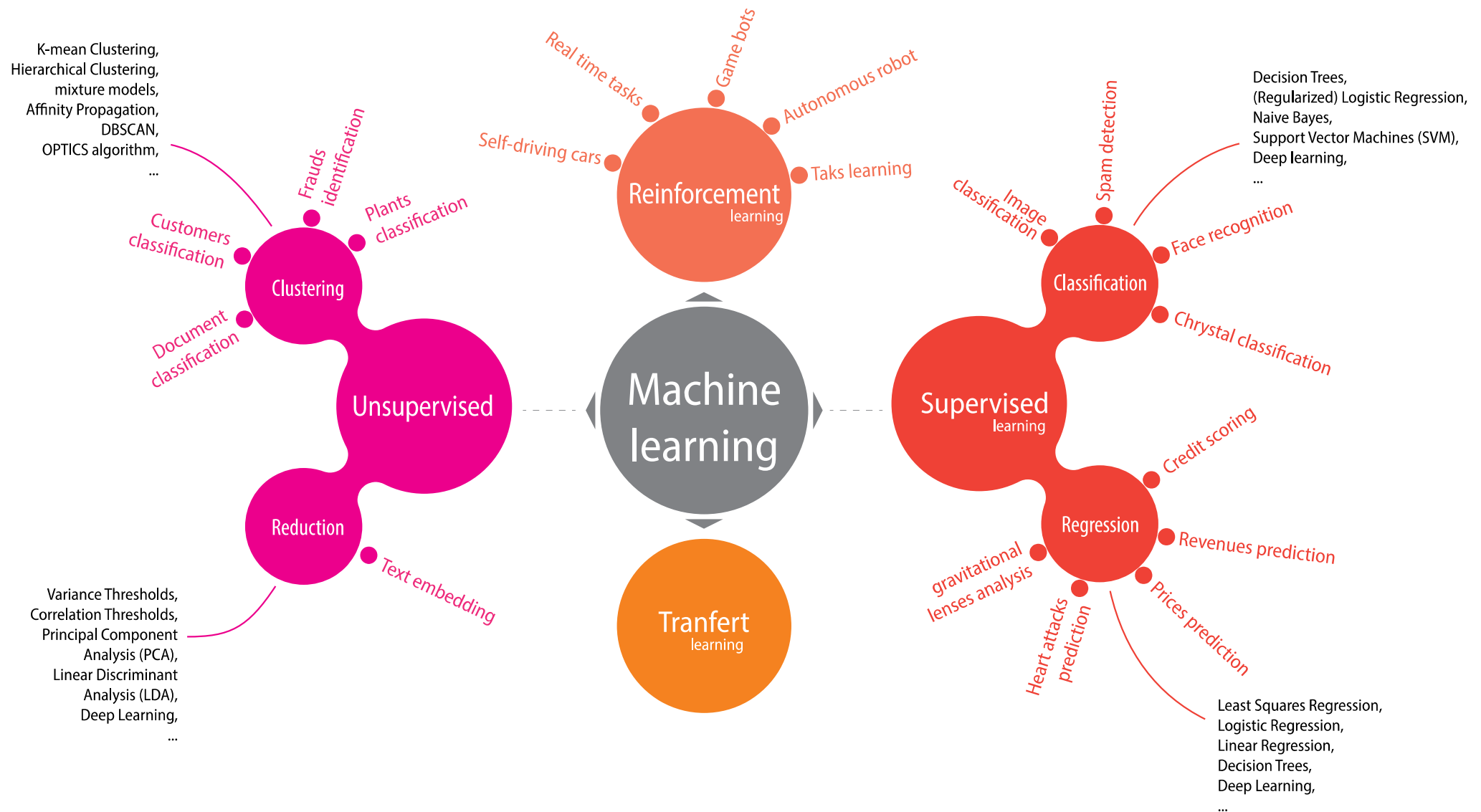
 **Résultat** : Mario finit par **terminer le jeu sans intervention humaine**, en apprenant les séquences d'actions optimales. [Démonstration vidéo](#)



Conclusion :

Le RL reflète un **aspect fondamental de l'intelligence humaine** : apprentissage **orienté par les buts**, via **essais, erreurs et ajustements**.

Science basée sur les données



Apprentissage en ligne : apprendre en continu



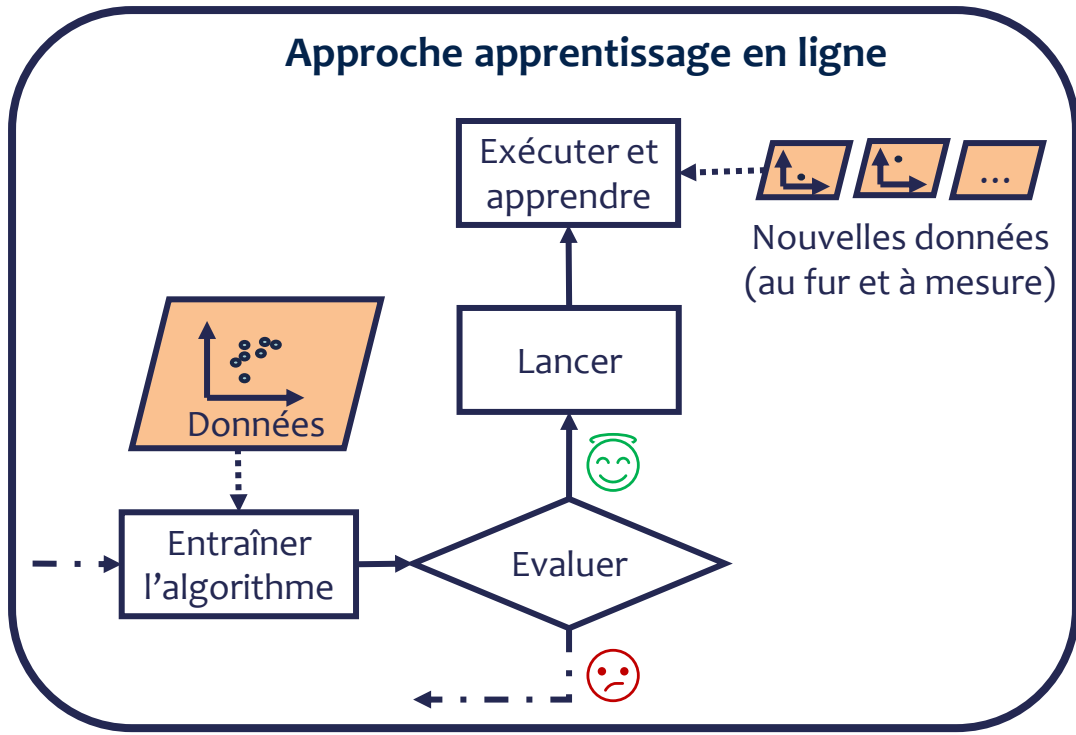
Comment fonctionne l'apprentissage en ligne ?

L'apprentissage en ligne (ou *online learning*) est un **mode d'apprentissage itératif et adaptatif** où le modèle est **entraîné progressivement**, au fur et à mesure que de **nouvelles données arrivent**.

Quel contexte convient à l'apprentissage en ligne ?

- Des systèmes recevant des données en flux continu et qui doivent s'adapter et évoluer rapidement de manière autonome.
- Des ressources informatiques limitées : lorsqu'un système a effectué son entraînement sur un nouveau jeu de données, il peut le supprimer → Economiser beaucoup de place.

Objectif : S'adapter en permanence à l'évolution des données et **minimiser l'erreur de prédiction** dans un environnement dynamique.



Un modèle est entraîné et lancé en production puis il continue à apprendre à mesure que de nouvelles données arrivent

Apprentissage hors mémoire : apprendre par lots

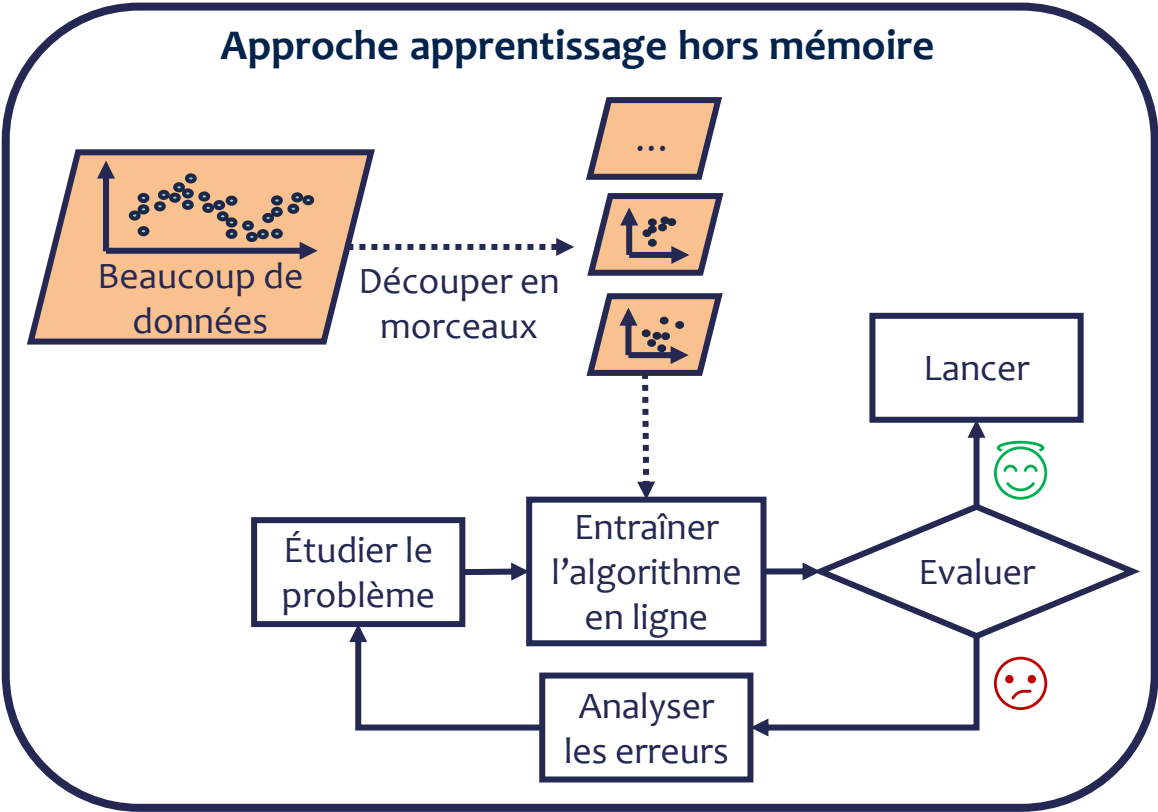


Quel est le principe de base ?

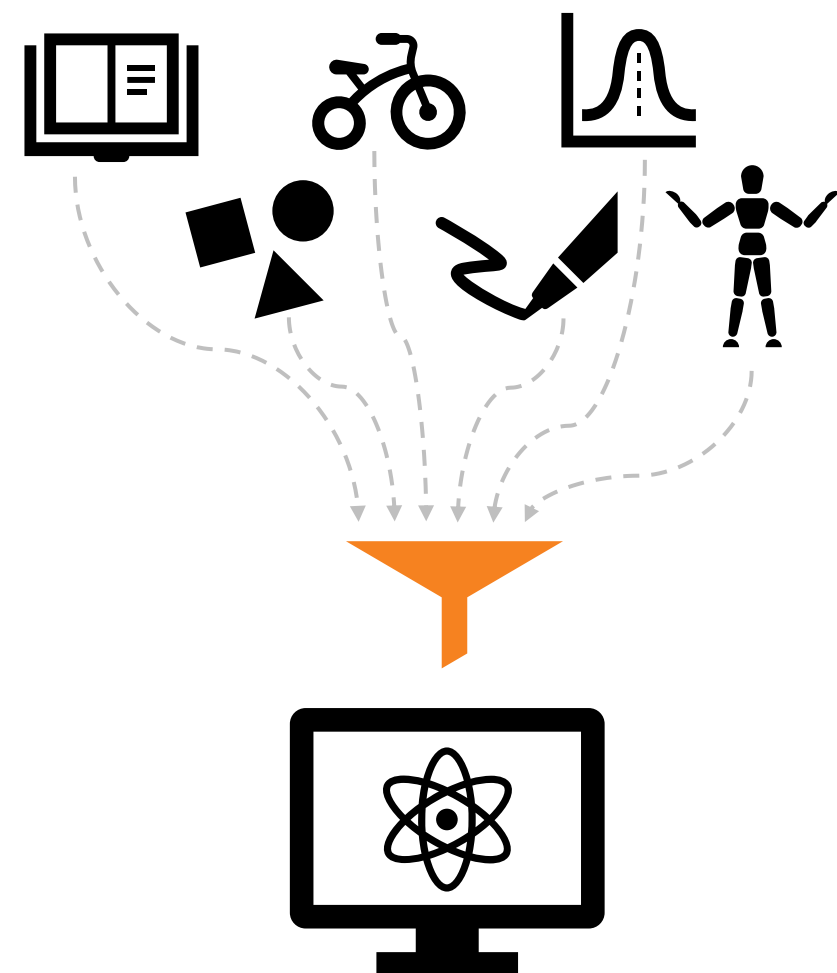
L'apprentissage hors mémoire consiste à **entraîner un modèle sur des données trop volumineuses** pour tenir en mémoire principale. Il est souvent réalisé grâce aux **algorithmes d'apprentissage en ligne**, capables de traiter les données **par petits lots successifs (mini-batches)**.

Quel est le fonctionnement ?

- Le système **charge une portion** du jeu de données.
- Il **met à jour le modèle** à partir de ces exemples.
- Il **libère la mémoire**, charge la portion suivante et **répète** le processus jusqu'à ce que toutes les données soient traitées.



Apprentissage à partir d'observation VS à partir d'un modèle



La plupart des tâches d'apprentissage automatique consistent à faire des prédictions.

- A partir d'un certain nombre d'exemples d'apprentissage, le système doit être capable de faire de bonnes prédictions pour des données qu'il n'a jamais vu : **il doit être capable de généraliser.**
- De bons résultats sur le jeu d'entraînement c'est bien mais insuffisant: ce qui est important c'est les résultats sur les nouvelles observations.

Apprentissage à partir d'observations (*instance-based learning*)



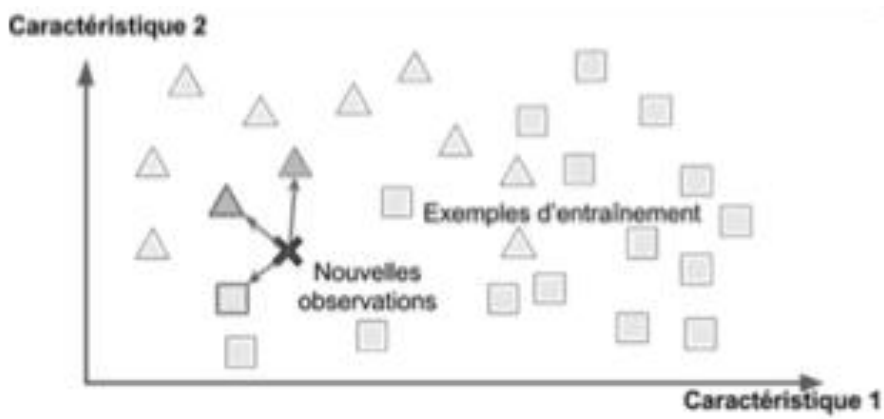
Quel est le principe de base ?

Le système apprend en **mémorisant les exemples d'apprentissage**, puis généralise en **comparant un nouvel exemple** aux données déjà observées à l'aide d'une **mesure de similarité**.

- Pas de modèle explicite construit à l'avance
- La prédiction repose sur les exemples les plus proches
- L'apprentissage est simple, mais la prédiction peut être coûteuse

Exemples d'algorithmes

- k-plus proches voisins (k-NN)
- Méthodes basées sur des distances ou similarités



La nouvelle observation va être classée comme triangle car la majorité des observations les plus semblables sont des triangles

Apprentissage à partir d'observations (*instance-based learning*)

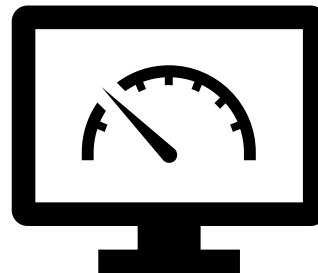


Exemples d'application : veut-on m'arnaquer ?

Le système **mémore** des emails déjà identifiés comme **spam** par l'utilisateur



Lorsqu'un nouvel email arrive, il est **comparé** aux spams connus



La similarité peut être mesurée par des critères simples, par exemple :

- nombre de mots en commun
- présence de termes caractéristiques des spams



Si l'email est **identique** ou **très similaire**, il est classé comme spam.



Le système ne construit pas de modèle général : il **compare directement aux exemples mémorisés**.

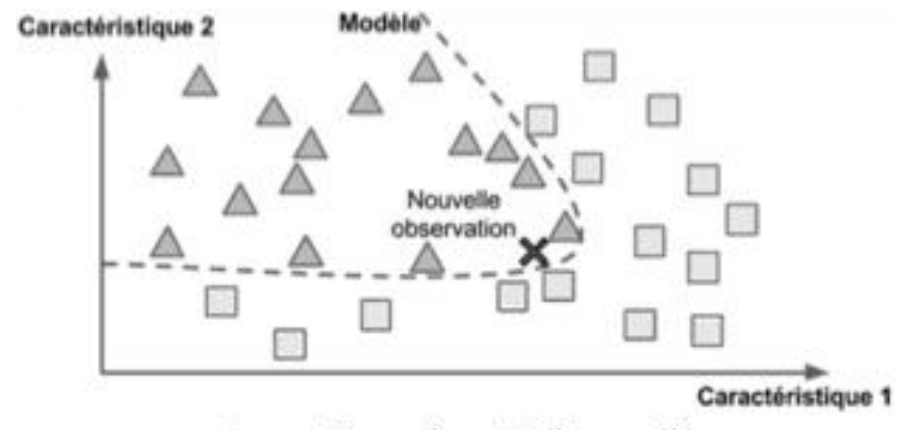
Apprentissage à partir d'un modèle (*model-based learning*)



Quel est le principe de base ?

Le système apprend en **construisant un modèle abstrait** à partir des données d'entraînement, puis utilise ce modèle pour **faire des prédictions sur de nouveaux cas**.

- Phase d'apprentissage explicite (entraînement du modèle)
- Prédiction rapide une fois le modèle appris
- Meilleure capacité de généralisation si le modèle est bien choisi



Apprentissage à partir d'un modèle

Exemples d'algorithmes

- Régression linéaire / logistique
- Arbres de décision
- Réseaux de neurones

Apprentissage à partir d'un modèle (*model-based learning*)



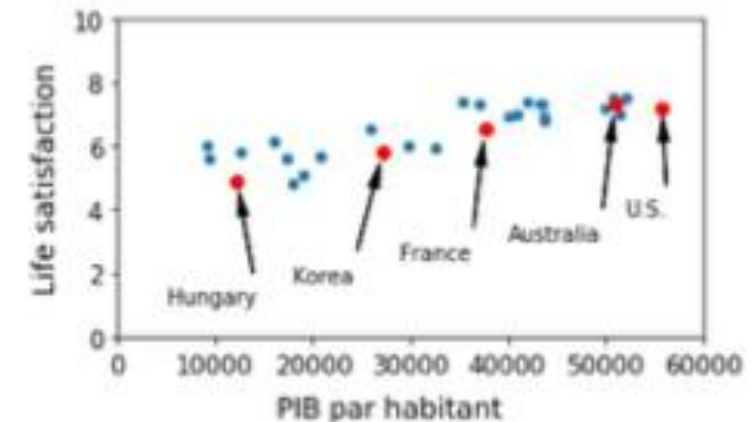
Exemples d'application : l'argent fait-il le bonheur ?

1. On télécharge les données fournies par l'OCDE (satisfaction individuelle).
2. On télécharge les données du PIB par habitant du FMI.
3. On représente sur un diagramme les données de quelques pays pris par hasard.

Il semble bien y avoir une tendance → quel modèle ?

- Bien que les données semblent entachées de bruit, on constate que le niveau de satisfaction augment à mesure que le PIB augmente.
- Le modèle de satisfaction individuelle est une **fonction linéaire** du PIB par habitant.
- On a fait la sélection du modèle.

Pays	PIB par habitant (USD)	Satisfaction individuelle
Hongrie	12 240	4,9
Corée	27 195	5,8
France	37 675	6,5
Australie	50 962	7,3
États-Unis	55 805	7,2



Apprentissage à partir d'un modèle (model-based learning)

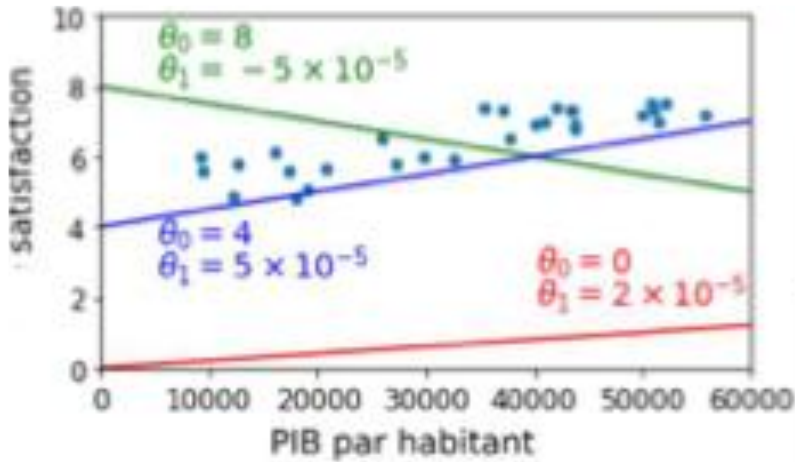


Exemples d'application : l'argent fait-il le bonheur ?

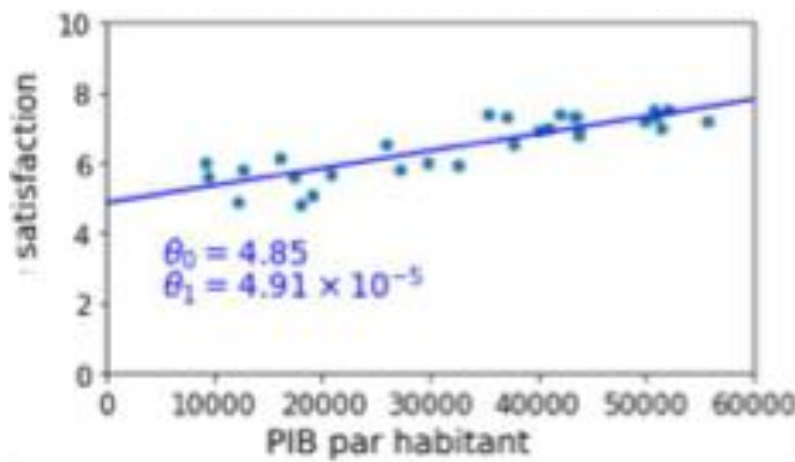
$satisfaction_{individuelle} = \theta_0 + \theta_1 \times PIB_{habitant}$

- On doit définir les valeurs des paramètres θ_0 et θ_1 .
- Quelles valeurs fourniront les meilleurs résultats ?
- On doit choisir une **fonction fitness** (fonction d'adaptation) dont les valeurs seront d'autant + élevées que le modèle est bon.
- En régression, on parle de **fonction de coût** : valeurs élevées équivalent à un modèle mauvais.
- En régression linéaire: la fct de coût mesure la distance entre les prédictions du modèle linéaire et les exemples d'entraînement.
- Algo de régression linéaire : on fournit les valeurs d'entraînement et il trouve les paramètres qui permettent d'ajuster au mieux le modèle linéaire aux données.
- On a fait l'entraînement du modèle.

Quelques modèles linéaires possibles



Modèle linéaire ajustant le mieux aux données



Apprentissage à partir d'un modèle (*model-based learning*)



Exemples d'application : l'argent fait-il le bonheur ?

$$satisfaction_{individuelle} = 4.85 + 4.91 \times 10^{-5} \times PIB_{habitant}$$

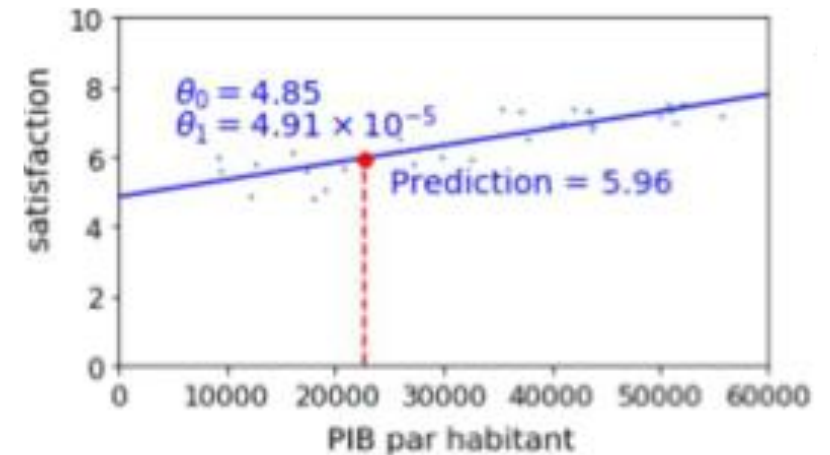
Si on veut connaître le degré de satisfaction personnelles des chypriotes :

- Les données de l'OCDE ne donnent pas de réponse
- On a le PIB par habitant de chypre 22 587 \$.
- On applique le modèle pour effectuer une bonne prédiction

$$\begin{aligned} satisfaction_{individuelle} &= 4.85 + 4.91 \times 10^{-5} \times 22\,587 \\ &= 5.96 \end{aligned}$$

Ci-dessous le code python qui charge les données, les prépare, les visualise sous forme d'un nuage de points, puis entraîne un modèle linéaire et effectue une prédiction.

Modèle linéaire ajustant le mieux aux données



Apprentissage à partir d'un modèle (*model-based learning*)



Exemples d'application : l'argent fait-il le bonheur ?

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import sklearn.linear_model

# Chargement des données
oecd_bli = pd.read_csv(datapath + "oecd_bli_2015.csv", thousands=',', encoding='utf-8')
gdp_per_capita = pd.read_csv(datapath + "gdp_per_capita.csv", thousands=',', delimiter='\t',
                             encoding='latin1', na_values="n/a")

# Préparation des données
country_stats = prepare_country_stats(oecd_bli, gdp_per_capita)
X = np.c_[country_stats["GDP per capita"]]
y = np.c_[country_stats["Life satisfaction"]]

# Visualisation des données
country_stats.plot(kind='scatter', x="GDP per capita", y="Life satisfaction")
plt.show()

# Sélection d'un modèle linéaire
model = sklearn.linear_model.LinearRegression()

# Entraînement du modèle
model.fit(X, y)

# Prédiction pour Chypre
X_new = [[22587]] # Chypre gdp_per_capita
print(model.predict(X_new)) # affichage [[ 5.96242338]]
``
```

Apprentissage à partir d'observations (*instance-based learning*)



Exemples d'application : l'argent fait-il le bonheur ?

- Si on veut utiliser un algorithme d'apprentissage basé sur les observations :
 - Le pays dont le PIB se rapproche le plus est la Slovénie 22 732
 - Le niveau de satisfaction des slovènes est de 5.7, on aurait prédit une satisfaction de 5.7 pour Chypre
- Algo des k plus proches voisins :
 - En prenant du recul : les 2 pays suivants les plus proches sont l'Espagne et le Portugal avec des niveaux de satisfaction de 5.1 et 6.5
 - La moyenne des 3 donne 5.77, là on a pris k=3 pour les k plus proches voisins

→ On remplace dans le code précédent :

```
import sklearn.linear_model
model = sklearn.linear_model.LinearRegression()
```

Par :

```
import sklearn.neighbors
model = sklearn.neighbors.KNeighborsRegressor(n_neighbors=3)
```

I. Fondamentaux du Deep Learning



Comprendre les concepts clés et les bases théoriques du Machine & Deep Learning

1.1

Introduction au Machine & Deep Learning : concepts, historique, applications

1.2

Types d'apprentissages en Machine & Deep Learning

1.3

Principales difficultés liées au Machine & Deep Learning

1.4

Présentation des frameworks : Scikit-learn, TensorFlow, PyTorch, Keras...

Difficultés générales

- **Quantité insuffisante de données d'apprentissage** → Le modèle ne parvient pas à apprendre des relations fiables.
- **Données d'entraînement non représentatives** → Le modèle fonctionne sur les données d'apprentissage mais échoue en situation réelle (biais).
- **Données de mauvaise qualité** → Valeurs manquantes, bruit, erreurs ou incohérences qui perturbent l'apprentissage.
- **Variables non pertinentes ou redondantes** → Le modèle apprend des informations inutiles ou trompeuses.
- **Sur-ajustement (overfitting)** → Le modèle mémorise les données d'entraînement et généralise mal sur de nouvelles données.
- **Sous-ajustement (underfitting)** → Le modèle est trop simple pour capturer la structure réelle des données.



Quantité insuffisante de données d'apprentissage

La plupart des algorithmes requièrent un grand nombre de données pour travailler correctement.

- Pour des données très simples il faut des milliers d'exemples
- Pour des données plus complexes, il faut des millions d'exemples

Pour les problèmes complexes, **les données sont plus importantes que les algorithmes**

Données de mauvaise qualité

- Consacrer du temps à nettoyer les données d'apprentissage dans les cas suivants :
- Supprimer les données aberrantes ou essayer de les corriger manuellement
- Gérer les valeurs manquantes dans les données

Données de mauvaise qualité: jeu de données rempli d'erreurs, de données aberrantes et de bruit)

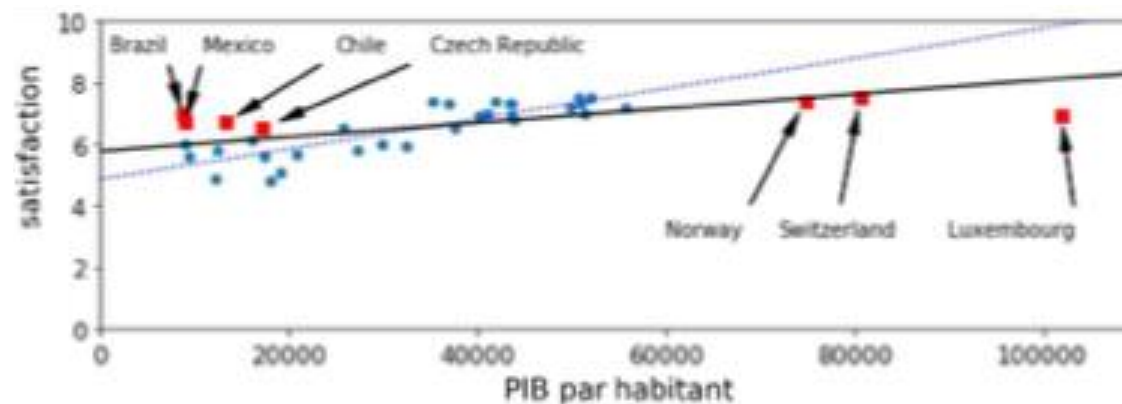
Données d'entraînement non représentatives

Il est essentiel que les données d'entraînement soient représentatives des nouveaux cas auxquels on veut généraliser.

Exemple : l'argent fait-il le bonheur ?

- L'ensemble des pays utilisé pour entraîner le modèle linéaire n'était pas parfaitement représentatif → **Le modèle linéaire ne convient pas.**
- L'ajout de pays manquants améliore le modèle.
- Données non représentatives → le modèle ne fera pas de bonnes prédictions pour les très pauvres ou très riches.

- Échantillon trop petit → **bruit d'échantillonnage**
- Échantillon trop grand, peut être non représentatif si la méthode d'échantillonnage est défectueuse → **biais d'échantillonnage**



Echantillon d'entraînement plus représentatif
Ligne continue = entraînement du modèle sur les nouvelles données Les pays très riches ne sont pas plus heureux que les moyennement riches
Certains pays pauvres semblent + heureux que les riches

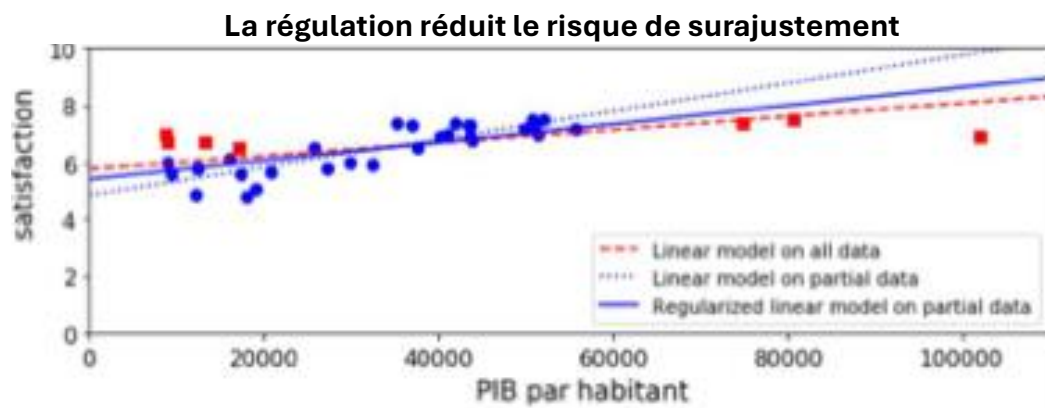
Variables non pertinentes ou redondantes

- Il faut que les données d'entraînement contiennent suffisamment de variables pertinentes et pas trop de non pertinentes.
- L'apprentissage automatique est basé sur **l'ingénierie de variables** (feature engineering):
 - **La sélection de variables** (parmi celles existantes, sur lesquelles il est plus utile de réaliser l'entraînement)
 - **L'extraction de variables** (combinant plusieurs existantes pour en produire une plus utile : algos de réduction de dimension)
 - **L'introduction de nouvelles variables** (la collecte de nouvelles données)

Si les données d'entrée sont mauvaises, la sortie le sera aussi « garbage in, garbage out »

Sur-ajustement (overfitting)

- La figure montre un modèle basé sur un polynôme de très haut degré qui surajuste fortement.
- Surajustement = modèle trop complexe par rapport à la quantité de données d'apprentissage et au bruit.
- **Quelles solutions ?**
 - **Simplifier le modèle:** sélectionner moins de paramètres en réduisant le nbr d'attributs des données ou en imposant des contraintes au modèle.
 - **Plus de données d'apprentissage**
 - **Réduire le bruit de ces données**(corriger les erreurs, supprimer les données aberrantes)



- Il faut trouver le bon équilibre entre un ajustement parfait des données d'entraînement et une simplicité du modèle garantissant une bonne généralisation.
- **Hyperparamètre :**
 - On contrôle le niveau de régularisation au niveau de l'apprentissage avec **un hyperparamètre**
 - C'est un paramètre de l'algo d'apprentissage et non du modèle.
 - Il n'est pas affecté à l'algo d'apprentissage : il est défini avant l'apprentissage et reste constant au cours de celui-ci.

Sous-ajustement (underfitting)

Sous-ajustement = modèle trop simple pour découvrir la structure sous-jacente des données

Dans l'exemple de satisfaction individuelle, le modèle linéaire risque de sous-ajuster les données.

Solutions possibles au problème:

- Choisir un modèle plus puissant (plus de paramètres)
- Fournir de meilleures variables à l'algorithme d'apprentissage (en les transformant au besoin).
- Réduire les contraintes sur le modèle (en réduisant l'hyperparamètre de régulation).

Difficultés spécifiques au Deep Learning

Le Deep Learning hérite des difficultés précédentes, mais ajoute des contraintes propres aux réseaux profonds :

1

Coût computationnel élevé :
Besoin de GPU / TPU, mémoire importante

2

Besoin massif de données :
Performances faibles avec peu de données (risque accru d'overfitting)

3

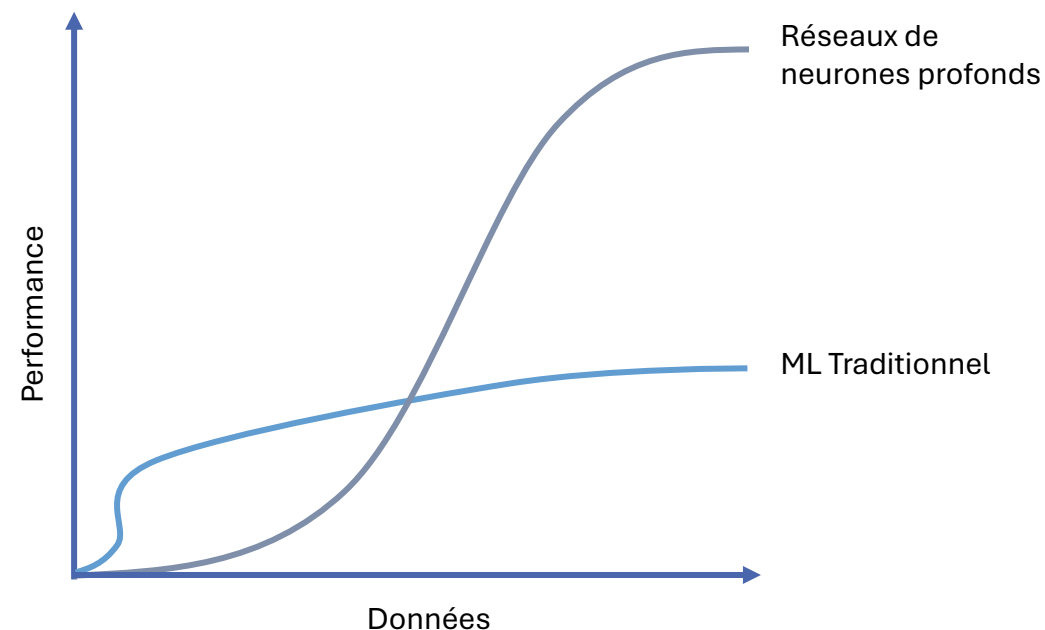
Complexité et interprétabilité :
modèles souvent considérés comme des *boîtes noires*

4

Temps d'entraînement long :
Optimisation complexe (descente de gradient, backpropagation)

5

Difficulté de réglage (hyperparamètres):
Architecture, nombre de couches, learning rate, régularisation...



Synthèse

Difficulté	Machine Learning	Deep Learning
Manque de données	Impact modéré	Impact très fort
Coût de calcul	Faible à modéré	Élevé
Interprétabilité	Souvent bonne	Souvent faible
Temps d'entraînement	Rapide	Long
Feature engineering	Important	Souvent appris automatiquement

Le **Deep Learning n'élimine pas les difficultés du Machine Learning**, il les **amplifie souvent**.

Il est particulièrement adapté lorsque :

- Les données sont massives
- Les relations sont complexes (images, texte, audio)

Sinon, un **modèle de Machine Learning** peut être plus efficace et plus simple.

I. Fondamentaux du Deep Learning



Comprendre les concepts clés et les bases théoriques du Machine & Deep Learning

1.1

Introduction au Machine & Deep Learning : concepts, historique, applications

1.2

Types d'apprentissages en Machine & Deep Learning

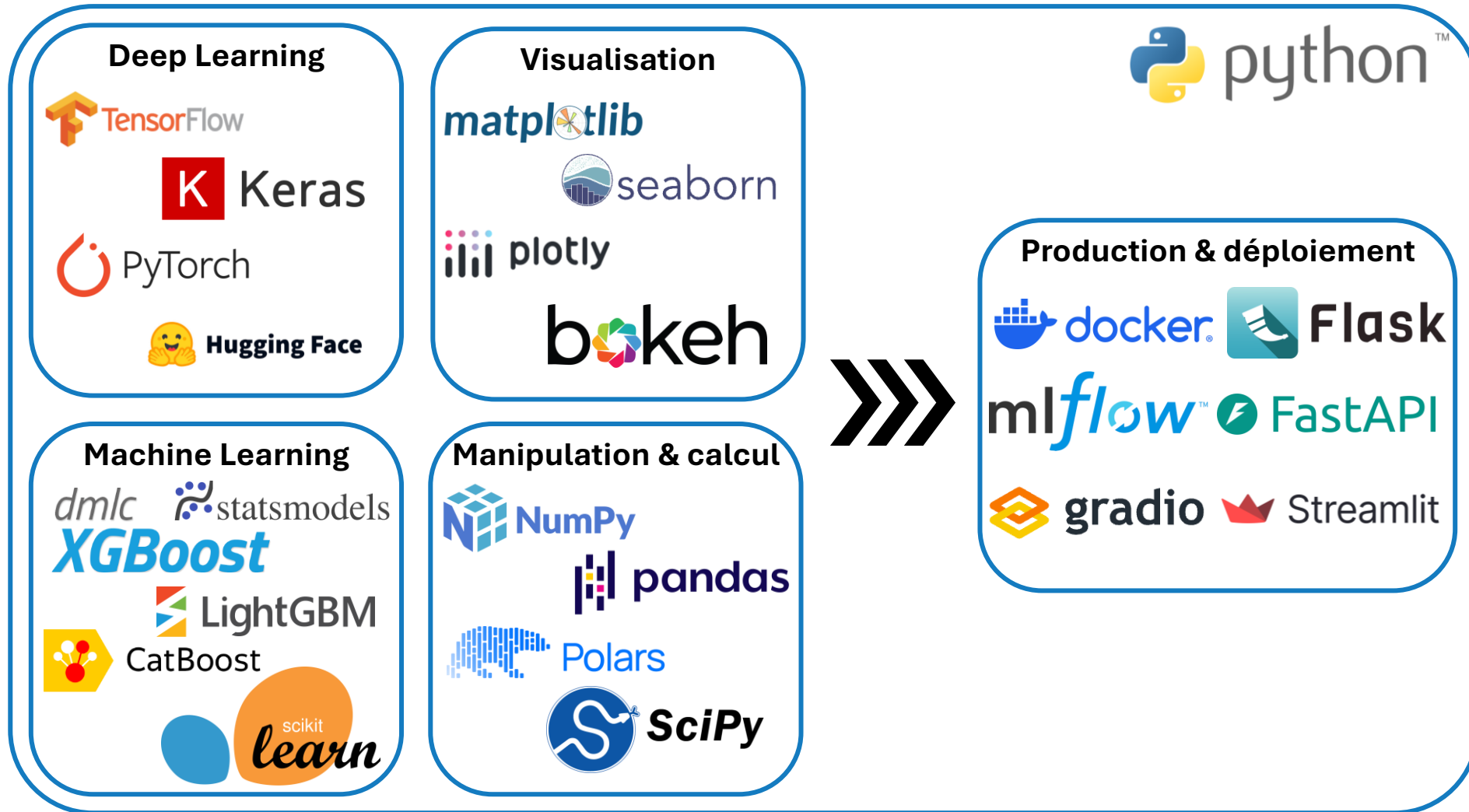
1.3

Principales difficultés liées au Machine & Deep Learning

1.4

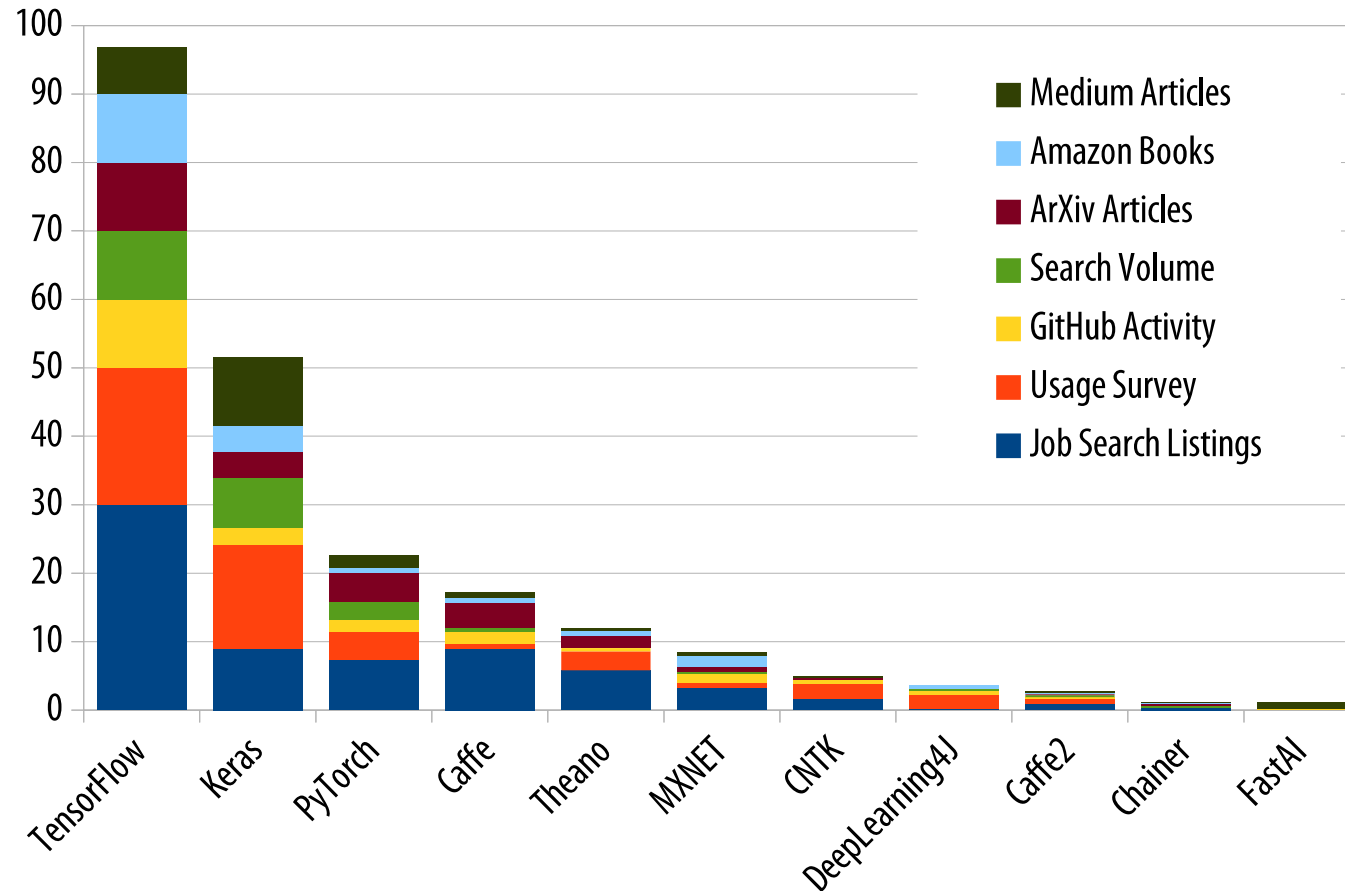
Présentation des frameworks : Scikit-learn, TensorFlow, PyTorch, Keras...

Un monde centré sur Python



Un monde centré sur Python

DL Framework Power Scores 2018



Framework DL le plus utilisé
Supporté par Google
API de bas niveau - une méthode difficile
Licence Apache



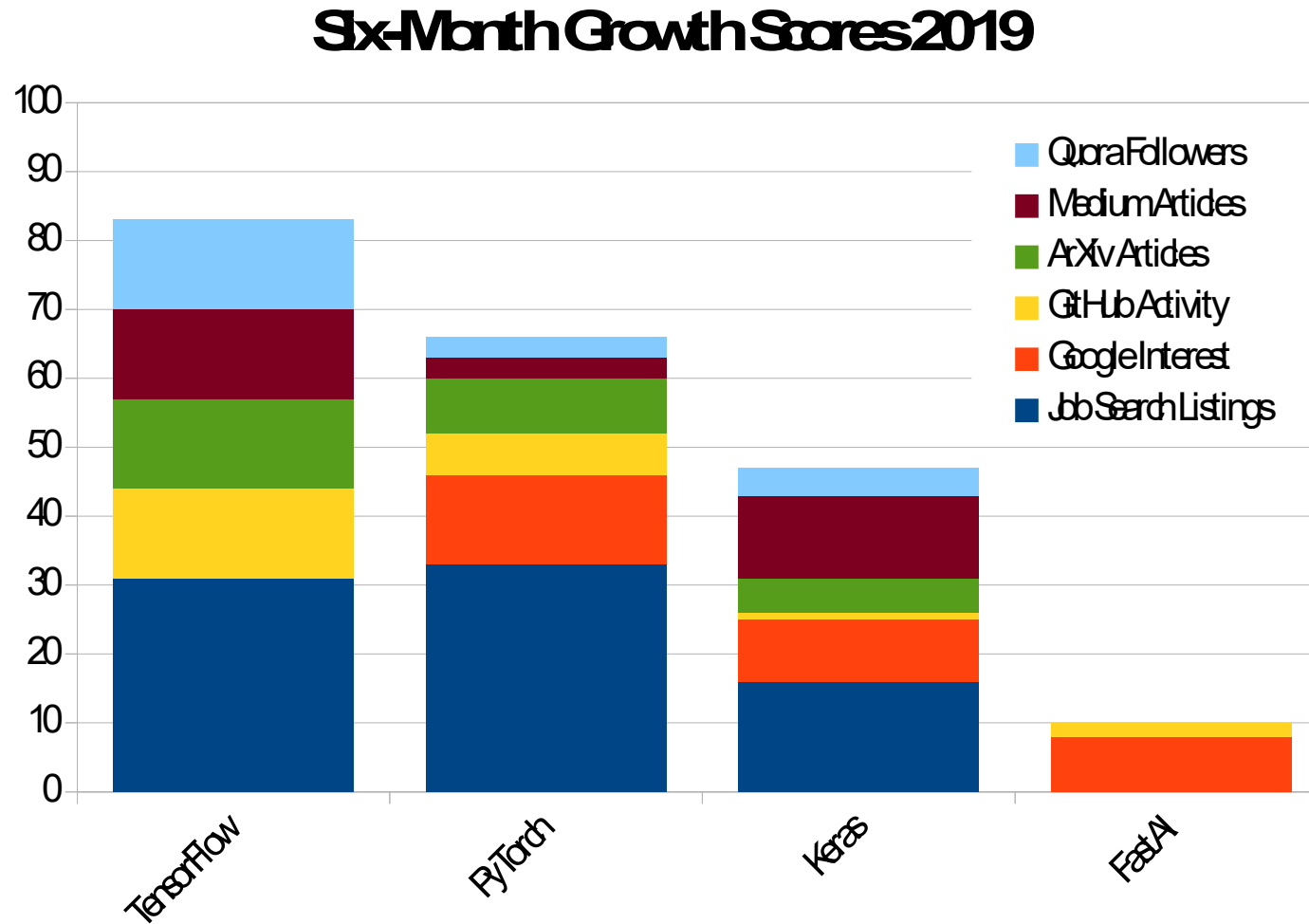
Par François Cholet (Google)
API de haut niveau
Partie sur TensorFlow depuis 2017
Licence MIT



De la bibliothèque Torch
Supporté par Facebook
Licence BSD

Source : Jeff Hale, « Deep Learning Framework Power Scores 2018 »

Un monde centré sur Python



 TensorFlow

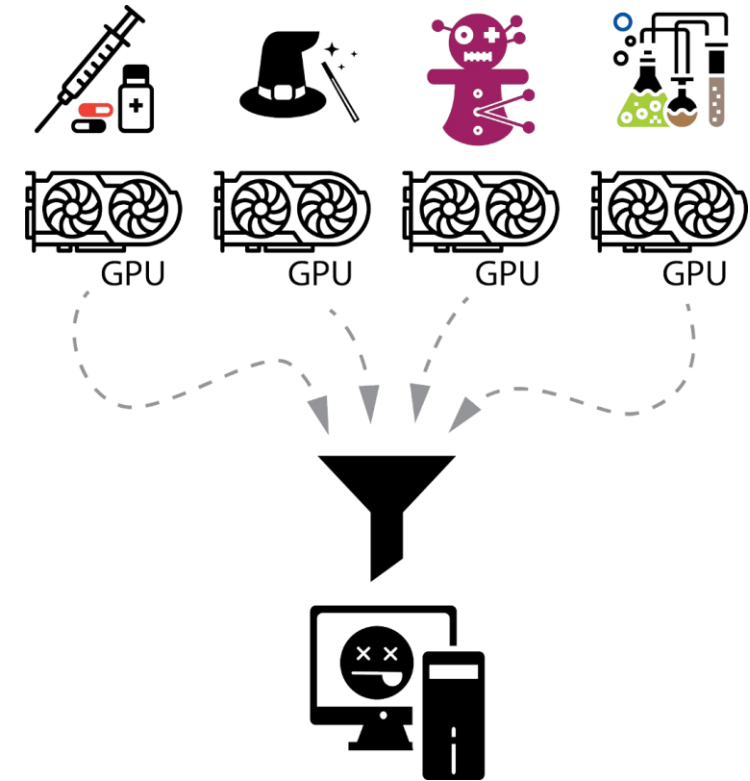
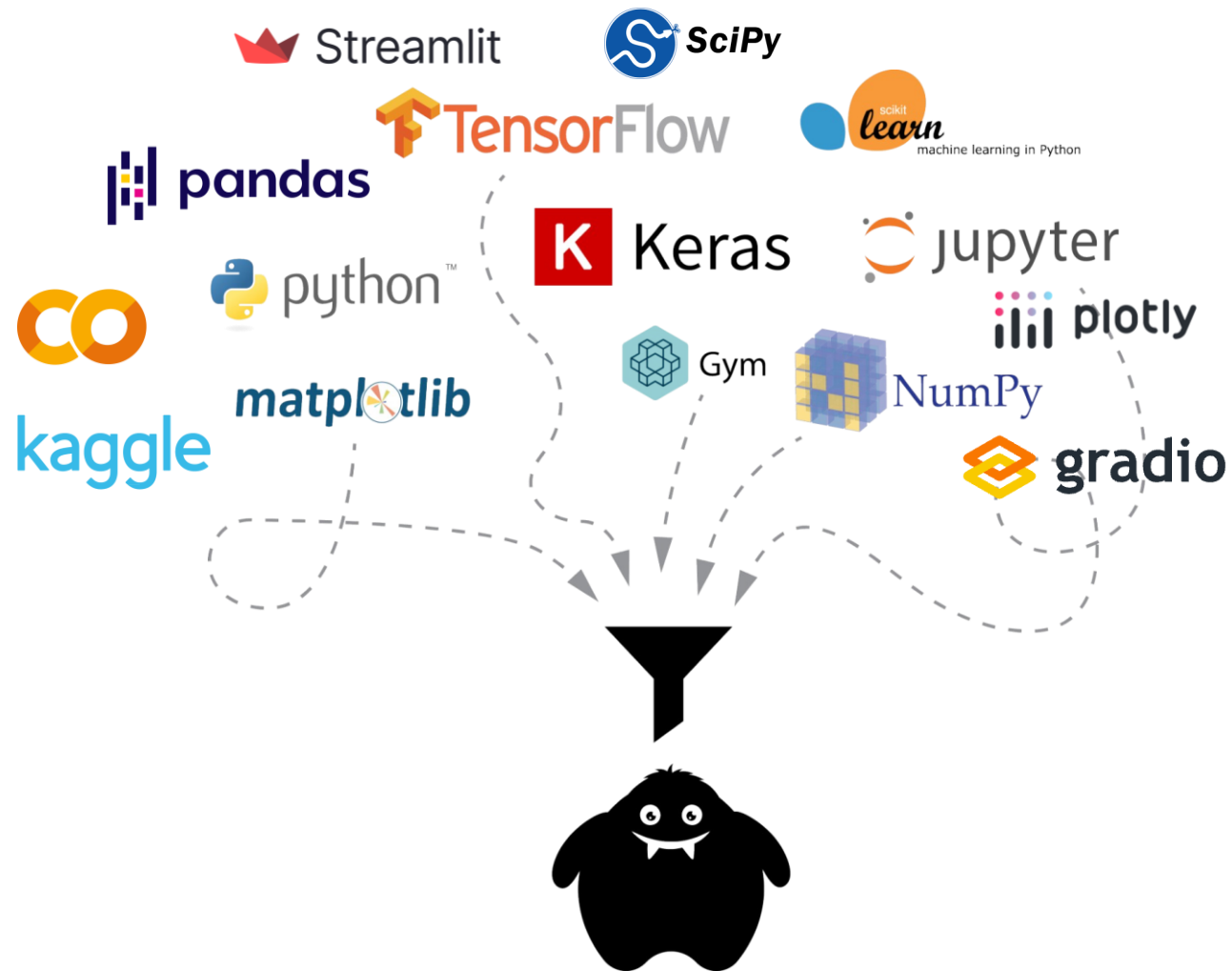
 PYTORCH

 Keras

 fast.ai

Source : Jeff Hale, « Deep Learning Framework Growth Scores 2019 »

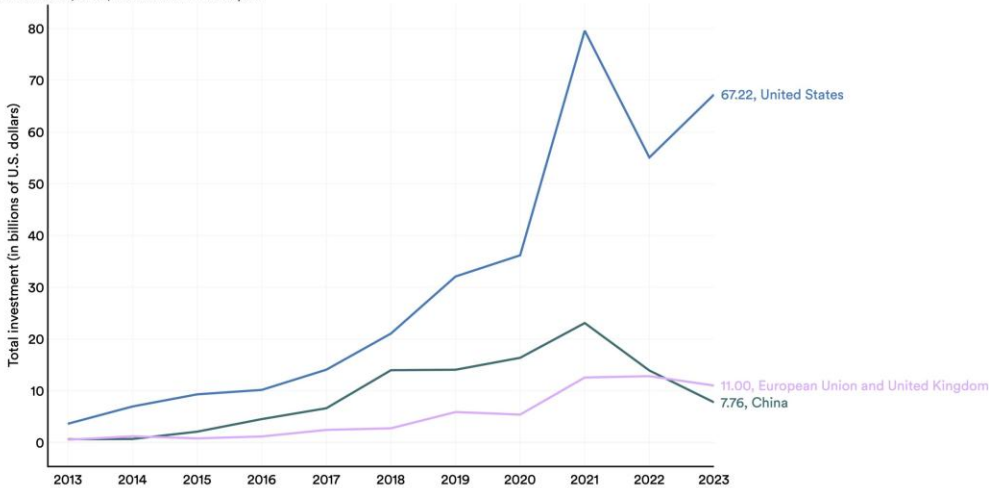
Une certaine complexité



Science en évolution

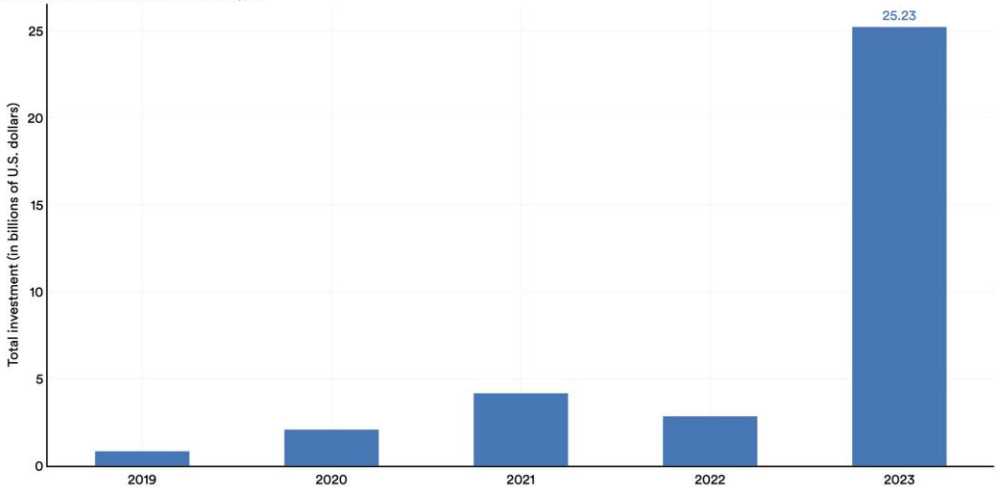
Private investment in AI by geographic area, 2013–23

Source: Quid, 2023 | Chart: 2024 AI Index report



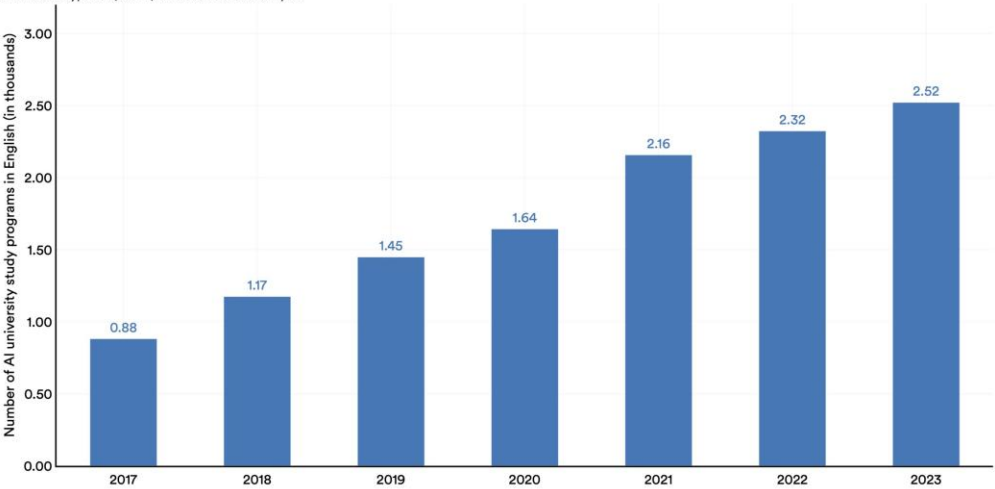
Private investment in generative AI, 2019–23

Source: Quid, 2023 | Chart: 2024 AI Index report



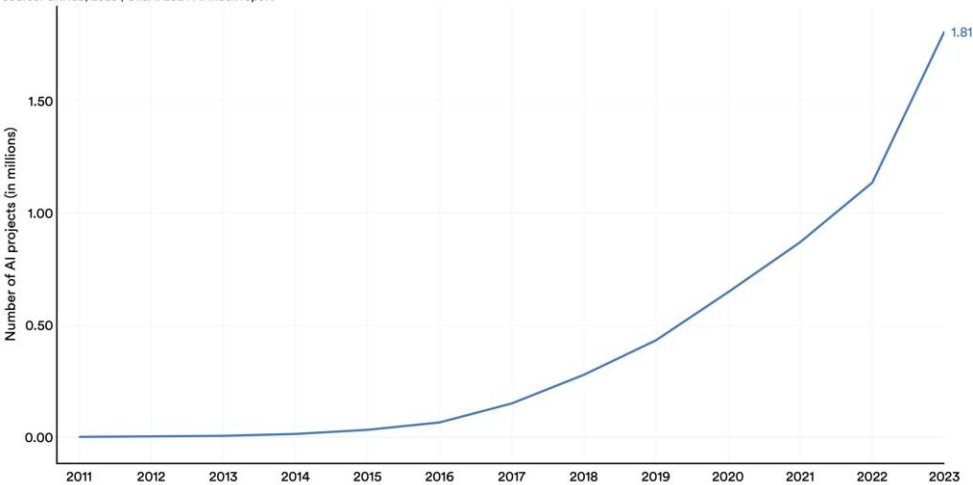
Number of AI university study programs in English in the world, 2017–23

Source: Studyportals, 2023 | Chart: 2024 AI Index report



Number of GitHub AI projects, 2011–23

Source: GitHub, 2023 | Chart: 2024 AI Index report

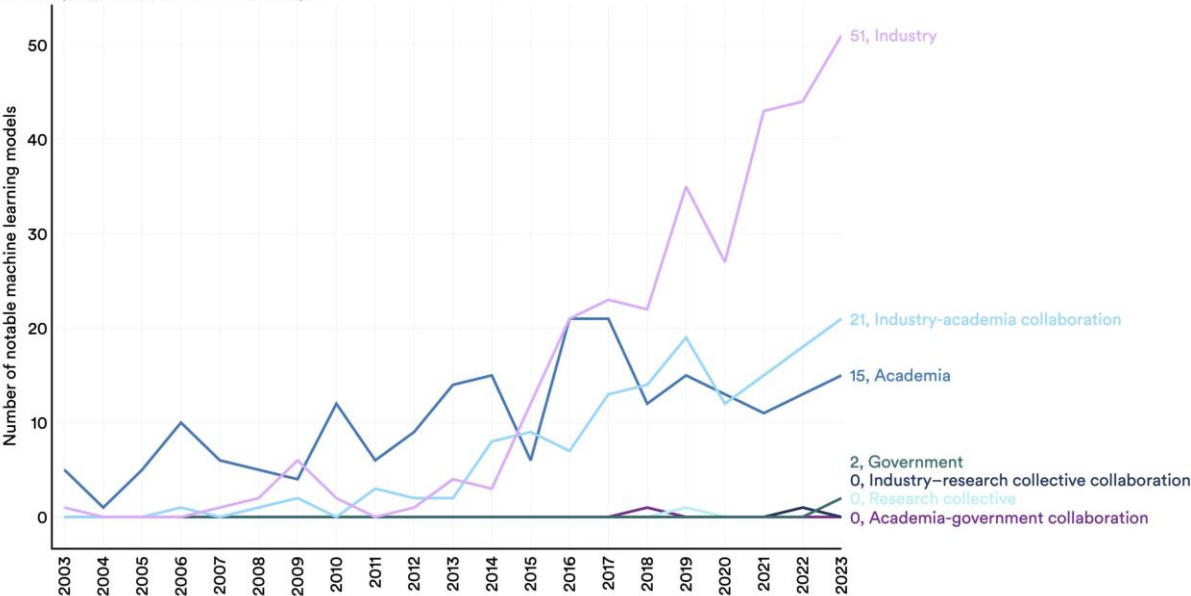


Source : AI Index 2024

Science en évolution

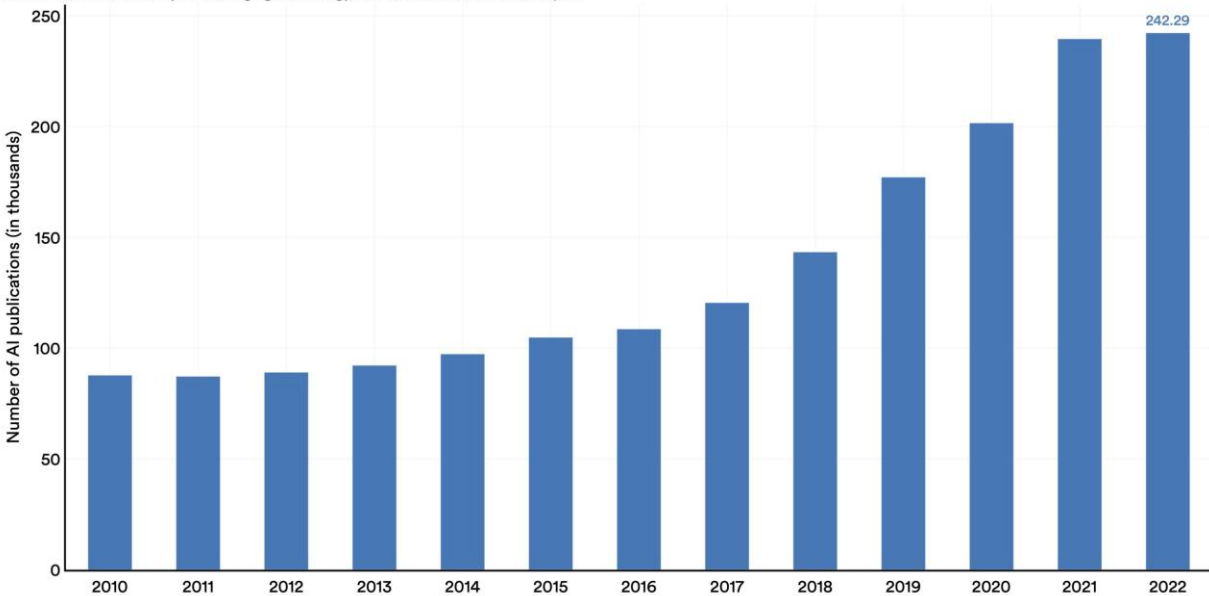
Number of notable machine learning models by sector, 2003–23

Source: Epoch, 2023 | Chart: 2024 AI Index report



Number of AI publications in the world, 2010–22

Source: Center for Security and Emerging Technology, 2023 | Chart: 2024 AI Index report

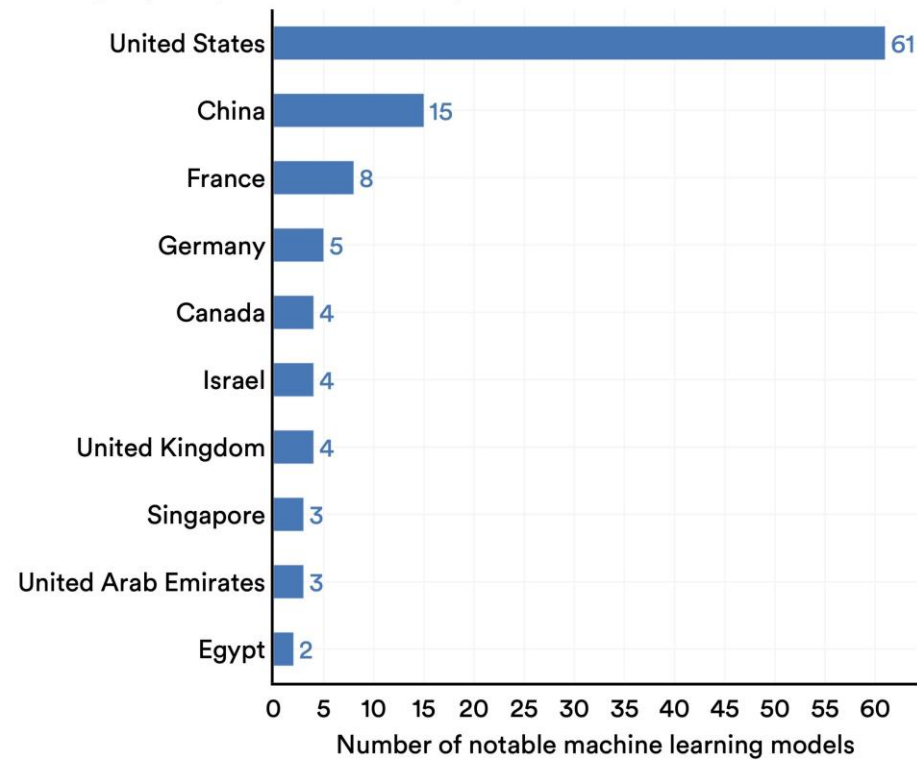


Source : AI Index 2024

Science en évolution

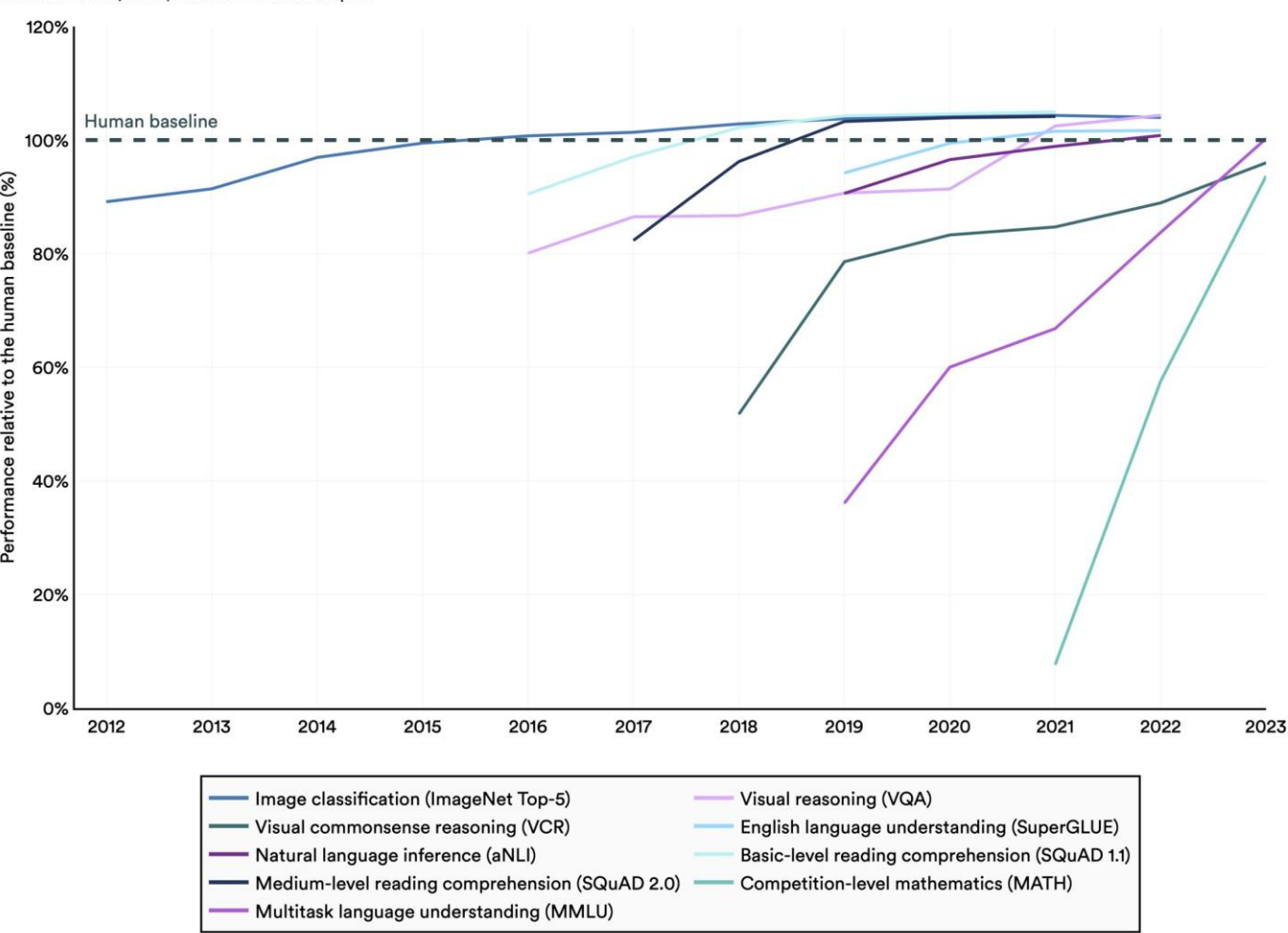
Number of notable machine learning models by geographic area, 2023

Source: Epoch, 2023 | Chart: 2024 AI Index report



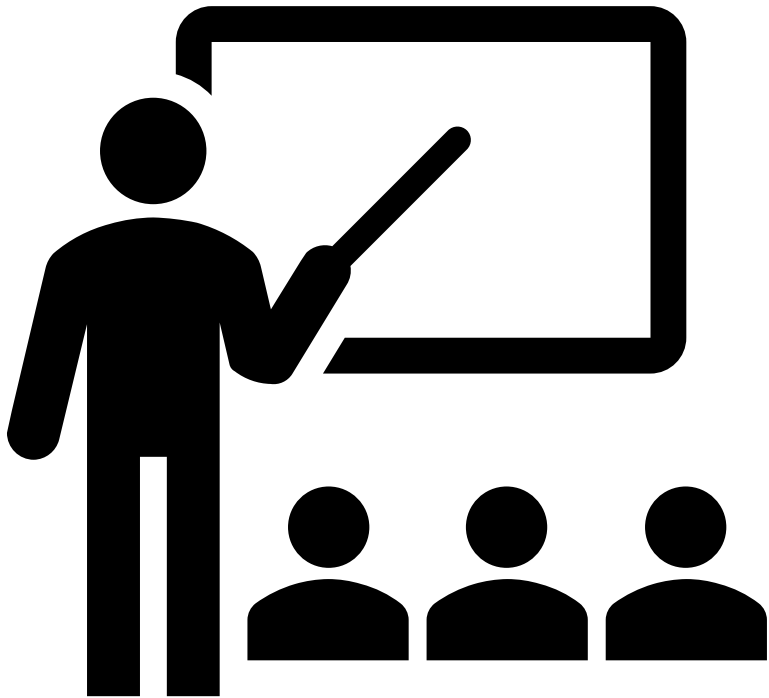
Select AI Index technical performance benchmarks vs. human performance

Source: AI Index, 2024 | Chart: 2024 AI Index report



Source : AI Index 2024

Contenu du cours



Fondamentaux du
Machine & Deep
Learning



Extraction, préparation,
visualisation et
communication des
données



Modélisation
supervisée et non-
supervisée en Machine
Learning



Modélisation
supervisée et avancée
en Deep Learning